



OTT 통합 플랫폼

캡스톤디자인 프로젝트 3조

2019xxxx 가xx
20212951 나xx
20212891 다xx



C O N T E N T S

1 프로젝트 개요

- 프로젝트 개요
- 프로젝트 선정배경
- 프로젝트 목표

2 중간발표 진행결과

- 중간발표 진행결과 요약
-

3 프로젝트 수행내용

- 프로젝트 수행 시나리오
- 개발환경 설계
- DB 테이블 구성
- 소스파일 구성
- 메뉴별 기능

4 프로젝트 성과

- 독창성 및 차별성
- 역할분담 및 수행내용
- 느낀점

5 결론 및 향후과제

- 결론
- 향후과제

프로젝트 개요

OTT 서비스 앱 사용자 및 사용률 23년 4월

앱	사용자 (만 명)	사용률
 넷플릭스	1,156	1위 63.6%
 쿠팡플레이	467	3위 43.6%
 티빙	411	2위 43.9%
 웨이브	293	21.5%
 Disney+	181	36.0%
 u+모바일tv	142	15.7%
 왓차	88	27.3%
 모바일B tv	59	23.1%

OTT서비스 통합

OTT 시장은 계속해서 성장하고 있으며, 사용자들의 요구도 점차 다양화되고 있다. 우리나라에서 흔히 사용되는 대표적인 OTT 서비스들을 하나의 플랫폼으로 통합하여 사용자가 더욱 편리하고 접근하기 쉽도록 OTT 서비스를 이용할 수 있는 것을 목표로 한다.

대표적인 OTT 서비스: Netflix, Disney+, Wavve, TVING, 왓차, 쿠팡플레이, Youtube Premium

프로젝트의 선정 배경

원하는 콘텐츠 찾기 번거로움

현재의 OTT 서비스는 각각 독립적으로 운영되며, 사용자는 여러 플랫폼을 이동하며 콘텐츠를 찾아야 한다. 사용자들은 콘텐츠를 찾는 과정에서 불편함을 겪고, 이용성이 저하된다. OTT 통합 플랫폼 서비스를 통해 다수의 OTT 서비스를 하나의 플랫폼으로 통합하여 제공함으로써, 사용자들이 여러 플랫폼을 번거롭게 이용하지 않고도 원하는 콘텐츠를 바로 찾고 시청할 수 있도록 한다.

콘텐츠 선택장애

다양한 선택지로 인해 무엇을 봐야 할지 결정하기가 어려워 흔히 OTT를 이용하는 대다수의 사용자가 콘텐츠를 결정하기가 어려워 결국 콘텐츠를 시청하지 못하는 경우가 많다. 추천 알고리즘을 적용하고 AI챗봇을 활용하여 개개인에게 맞는 콘텐츠 추천을 해줌으로써 사용자가 어떤 콘텐츠를 시청할지 결정하기 쉽도록 개선해 줄 수 있다.

프로젝트 목표

OTT 플랫폼의 증가에 따라 사용자가 여러 OTT 플랫폼을 각각 관리해야 하는 불편함을 줄이기 위해 각각의 OTT 플랫폼을 하나의 서비스로 관리할 수 있도록 하는 OTT 통합 플랫폼을 개발하고 OTT 통합 검색, 콘텐츠 추천, 커뮤니티, AI 챗봇과 같은 기능을 제공하는 것을 목표로 한다.



중간발표 진행결과요약

Django 프로젝트 생성

MySQL연동 및 DB 구축

TMDB의 Open API를 통한 데이터 수집

UI 설계 및 구현

스트리밍 제공업체의 범위 조절

로그인,회원가입,검색 기능 구현

상세페이지/컨텐츠 정보 연결

데이터 수집 범위 수정

프로젝트 수행 내용 요약

 HUBFLIX

HUBFLIX

데이터 수집

TMDB Open API
데이터 수집DB구축 및 데이터
저장

기능 구현

로그인, 회원가입

회원정보 수정

검색, 상세정보

AI챗봇 구현

AI 챗봇 장고 프로젝
트 연결

대화형으로 구현

서버

AWS EC2

Gunicorn 연동

Ngnix 연동

개인화된 추천
알고리즘 구현

장르별 추천

단순 필터, 정렬을 이
용한 콘텐츠 노출

프로젝트 수행 내용

시스템 모듈 상세 설계 [개발환경 설계]

-개발언어:Python, HTML, CSS, JavaScript, SQL

-서버 소프트웨어: Nginx, gunicorn

-서버 프레임워크: Django

-주요 라이브러리: Python Selenium Library

-서버 프로세서: AWS EC2

-DBMS: MySQL

-버전관리:Github

-운영체제:Window

시나리오 구성요소

-사용자(유저)

-서버: 연결,데이터 수집 연산

-DB: 수집 데이터 저장

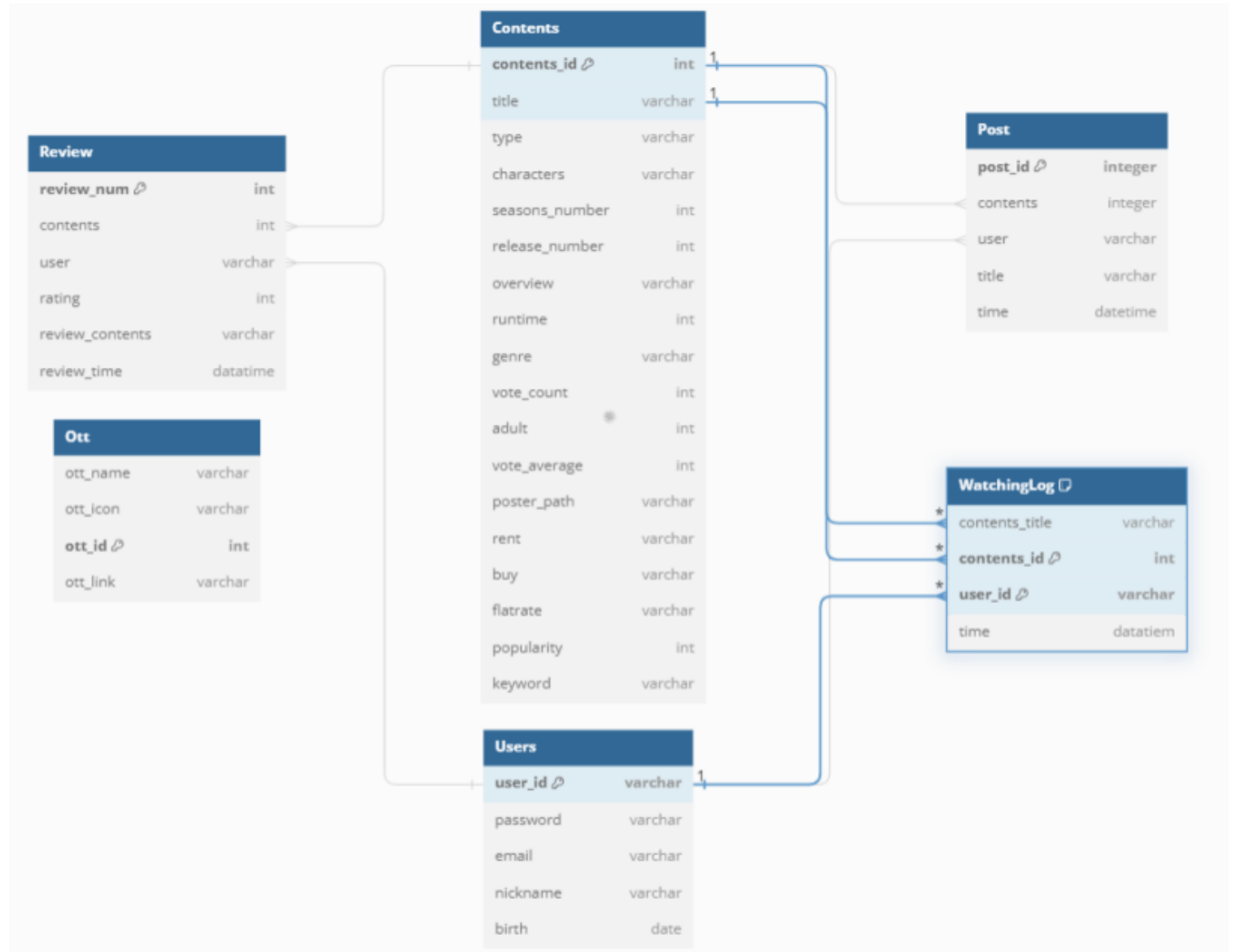
-외부 OTT

UI 디자인툴 : Figma 사용



데이터베이스

[DB테이블 구성]



프로젝트 수행내용

데이터 수집 방법

```

1 import requests
2 import json
3
4 # 攻击 url
5 url_key = "7f28b799319016406049732026785"
6
7 headers = {
8     "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4328.220 Safari/537.36"
9 }
10
11 # 攻击 url
12 url_key = "7f28b799319016406049732026785"
13
14 # 攻击 url
15 url_key = "7f28b799319016406049732026785"
16
17 # 攻击 url
18 url_key = "7f28b799319016406049732026785"
19
20 # 攻击 url
21 url_key = "7f28b799319016406049732026785"
22
23 # 攻击 url
24 url_key = "7f28b799319016406049732026785"
25
26 # 攻击 url
27 url_key = "7f28b799319016406049732026785"
28
29 # 攻击 url
30 url_key = "7f28b799319016406049732026785"
31
32 # 攻击 url
33 url_key = "7f28b799319016406049732026785"
34
35 # 攻击 url
36 url_key = "7f28b799319016406049732026785"
37
38 # 攻击 url
39 url_key = "7f28b799319016406049732026785"
40
41 # 攻击 url
42 url_key = "7f28b799319016406049732026785"
43
44 # 攻击 url
45 url_key = "7f28b799319016406049732026785"
46
47 # 攻击 url
48 url_key = "7f28b799319016406049732026785"
49
50 # 攻击 url
51 url_key = "7f28b799319016406049732026785"
52
53 # 攻击 url
54 url_key = "7f28b799319016406049732026785"
55
56 # 攻击 url
57 url_key = "7f28b799319016406049732026785"
58
59 # 攻击 url
60 url_key = "7f28b799319016406049732026785"
61
62 # 攻击 url
63 url_key = "7f28b799319016406049732026785"
64
65 # 攻击 url
66 url_key = "7f28b799319016406049732026785"
67
68 # 攻击 url
69 url_key = "7f28b799319016406049732026785"
70
71 # 攻击 url
72 url_key = "7f28b799319016406049732026785"
73
74 # 攻击 url
75 url_key = "7f28b799319016406049732026785"
76
77 # 攻击 url
78 url_key = "7f28b799319016406049732026785"
79
80 # 攻击 url
81 url_key = "7f28b799319016406049732026785"
82
83 # 攻击 url
84 url_key = "7f28b799319016406049732026785"
85
86 # 攻击 url
87 url_key = "7f28b799319016406049732026785"
88
89 # 攻击 url
90 url_key = "7f28b799319016406049732026785"
91
92 # 攻击 url
93 url_key = "7f28b799319016406049732026785"
94
95 # 攻击 url
96 url_key = "7f28b799319016406049732026785"
97
98 # 攻击 url
99 url_key = "7f28b799319016406049732026785"
100
101 # 攻击 url
102 url_key = "7f28b799319016406049732026785"
103
104 # 攻击 url
105 url_key = "7f28b799319016406049732026785"
106
107 # 攻击 url
108 url_key = "7f28b799319016406049732026785"
109
110 # 攻击 url
111 url_key = "7f28b799319016406049732026785"
112
113 # 攻击 url
114 url_key = "7f28b799319016406049732026785"
115
116 # 攻击 url
117 url_key = "7f28b799319016406049732026785"
118
119 # 攻击 url
120 url_key = "7f28b799319016406049732026785"
121
122 # 攻击 url
123 url_key = "7f28b799319016406049732026785"
124
125 # 攻击 url
126 url_key = "7f28b799319016406049732026785"
127
128 # 攻击 url
129 url_key = "7f28b799319016406049732026785"
129
130 # 攻击 url
131 url_key = "7f28b799319016406049732026785"
132
133 # 攻击 url
134 url_key = "7f28b799319016406049732026785"
135
136 # 攻击 url
137 url_key = "7f28b799319016406049732026785"
138
139 # 攻击 url
140 url_key = "7f28b799319016406049732026785"
141
142 # 攻击 url
143 url_key = "7f28b799319016406049732026785"
144
145 # 攻击 url
146 url_key = "7f28b799319016406049732026785"
147
148 # 攻击 url
149 url_key = "7f28b799319016406049732026785"
150
151 # 攻击 url
152 url_key = "7f28b799319016406049732026785"
153
154 # 攻击 url
155 url_key = "7f28b799319016406049732026785"
156
157 # 攻击 url
158 url_key = "7f28b799319016406049732026785"
159
160 # 攻击 url
161 url_key = "7f28b799319016406049732026785"
162
163 # 攻击 url
164 url_key = "7f28b799319016406049732026785"
165
166 # 攻击 url
167 url_key = "7f28b799319016406049732026785"
168
169 # 攻击 url
170 url_key = "7f28b799319016406049732026785"
171
172 # 攻击 url
173 url_key = "7f28b799319016406049732026785"
174
175 # 攻击 url
176 url_key = "7f28b799319016406049732026785"
177
178 # 攻击 url
179 url_key = "7f28b799319016406049732026785"
180
181 # 攻击 url
182 url_key = "7f28b799319016406049732026785"
183
184 # 攻击 url
185 url_key = "7f28b799319016406049732026785"
186
187 # 攻击 url
188 url_key = "7f28b799319016406049732026785"
189
190 # 攻击 url
191 url_key = "7f28b799319016406049732026785"
192
193 # 攻击 url
194 url_key = "7f28b799319016406049732026785"
195
196 # 攻击 url
197 url_key = "7f28b799319016406049732026785"
198
199 # 攻击 url
200 url_key = "7f28b799319016406049732026785"
201
202 # 攻击 url
203 url_key = "7f28b799319016406049732026785"
204
205 # 攻击 url
206 url_key = "7f28b799319016406049732026785"
207
208 # 攻击 url
209 url_key = "7f28b799319016406049732026785"
210
211 # 攻击 url
212 url_key = "7f28b799319016406049732026785"
213
214 # 攻击 url
215 url_key = "7f28b799319016406049732026785"
216
217 # 攻击 url
218 url_key = "7f28b799319016406049732026785"
219
220 # 攻击 url
221 url_key = "7f28b799319016406049732026785"
222
223 # 攻击 url
224 url_key = "7f28b799319016406049732026785"
225
226 # 攻击 url
227 url_key = "7f28b799319016406049732026785"
228
229 # 攻击 url
230 url_key = "7f28b799319016406049732026785"
231
232 # 攻击 url
233 url_key = "7f28b799319016406049732026785"
234
235 # 攻击 url
236 url_key = "7f28b799319016406049732026785"
237
238 # 攻击 url
239 url_key = "7f28b799319016406049732026785"
240
241 # 攻击 url
242 url_key = "7f28b799319016406049732026785"
243
244 # 攻击 url
245 url_key = "7f28b799319016406049732026785"
246
247 # 攻击 url
248 url_key = "7f28b799319016406049732026785"
249
250 # 攻击 url
251 url_key = "7f28b799319016406049732026785"
252
```

인기순

1. TMDb API에서 인기 있는 콘텐츠를
pageNum 값으로 요청
2. 출력 결과의 movie_ID를 저장하여 상세정
보, 키워드, 등장인물의 API로 각각 요청하
여 값 저장
3. 장르, 키워드 등장인물은 값이 여러 개 이므
로 ','로 구분하여 하나의 문자열로 처리하여
저장
4. 요청한 데이터들을 딕셔너리 형태로 저장하
여 json 파일을 open하여 저장
5. 저장한 json 파일을 Django 프로젝트에서
명령어를 통해 DB로 반영

```

1  import requests
2  import json
3
4  # URL to the API endpoint
5  url = "http://10.10.10.10:8080/api/v1/users"
6
7  # Headers for the request
8  headers = {
9      "Content-Type": "application/json",
10     "Authorization": "Bearer 404059752626780"
11 }
12
13 # Data to be sent in the request body
14 data = {
15     "username": "john.doe",
16     "password": "password123",
17     "email": "john.doe@example.com"
18 }
19
20 # Send the POST request
21 response = requests.post(url, headers=headers, json=data)
22
23 # Check the status of the response
24 status_code = response.status_code
25
26 # Print the response
27 print(f"Status Code: {status_code}")
28
29 # If the status code is 200 (Success), print the response body
30 if status_code == 200:
31     # Parse the JSON response
32     user_data = response.json()
33
34     # Print the user data
35     print(f"User Data: {user_data}")
36
37 # If the status code is not 200, print an error message
38 else:
39     print(f"Error: {status_code} - {response.text}")
40
41 # Close the session
42 session.close()

```

moive id를 증가, 반복

1. TMDb API에서 movie_ID를 1부터 반복하여 상세정보, 키워드, 등장인물의 API로 각각 요청하여 값 저장
2. 장르, 키워드 등장인물은 값이 여러 개 이므로 ','로 구분하여 하나의 문자열로 처리하여 저장
3. 요청한 데이터들을 딕셔너리 형태로 저장하여 json 파일을 open하여 저장
4. 저장한 json 파일을 Django 프로젝트에서 명령어를 통해 DB로 반영

프로젝트 수행내용

Django project

파일 내용 구성

```

▼ django \ Hubflix
  ▼ app
    > __pycache__
    > fixtures
    > migrations
    > static
    > templates
    • __init__.py
    • admin.py
    • apps.py
    • forms.py
    • models.py
    • recommend.py
    • serializers.py
    • tests.py
    • urls.py
    • views.py
  ▼ Hubflix
    > __pycache__
    • __init__.py
    • asgi.py
    • settings.py
    • urls.py
    • wsgi.py
    ≡ db.sqlite3
    • manage.py
    ≡ requirements.txt
  
```

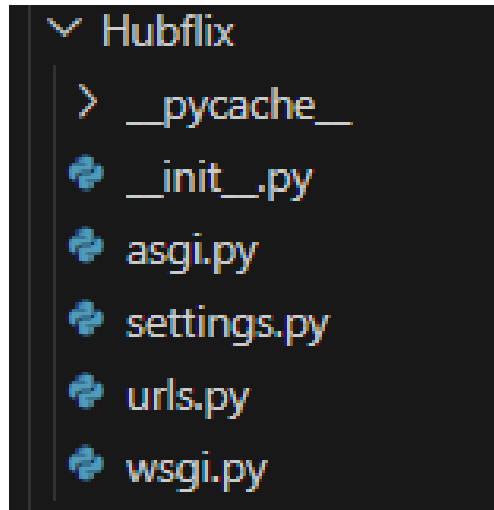
Django는 하나의 프로젝트에 여러 APP이 존재 가능하며
APP별로 독단적으로 개발이 가능하고 여러 APP을 통합할 수 있다.

- django : Django 프로젝트의 루트 폴더
- └ app : 생성한 Django 애플리케이션 폴더
- └ Hubflix : 생성한 Django 프로젝트를 구성하는 핵심 파일이 있는 폴더
- └ db.sqlite3 : Django 프로젝트에서 사용하는 기본 DB
(본 프로젝트에서는 MySQL을 사용)
- └ manage.py : Django에서 지원하는 프로젝트를 관리하기 위한
다양한 명령어를 실행할 수 있는 파일
- └ requirements.txt : 가상환경에서 pip install로 설치한 목록 텍스트 파일

프로젝트 수행내용

Django project

파일 내용 구성



Hubflix

- └ __pycache__ : Python을 컴파일하여 실행할 수 있도록 준비한 바이트 파일들이 저장된 폴더 (프로그램 실행 속도를 높여주기 위한 파일)
- └ __init__.py : Python 패키지로 인식하기 위한 파일
- └ asgi.py : ASGI(Asynchronous Server Gateway Interface) 서버 진입점 (asgi 사용 시 사용되는 파일)
- └ settings.py : Django 프로젝트의 설정 파일, 데이터베이스, 템플릿, 정적 파일, 사용할 애플리케이션 등의 설정을 저장하는 파일
- └ urls.py : Django 프로젝트의 URL 패턴을 정의하는 파일
- └ wsgi.py : WSGI(Web Server Gateway Interface) 서버 진입점 (wsgi 사용 시 사용되는 파일)

* __pycache__는 파이썬 스크립트 실행 시 자동으로 생긴 디렉토리

* __init__.py, asgi.py, settings.py, urls.py, wsgi.py는

Django 프로젝트 생성 시 자동으로 생성된 파일

프로젝트 수행내용

Django project

파일 내용 구성

```

  app
  ├── __pycache__
  ├── fixtures
  ├── migrations
  ├── static
  ├── templates
  ├── __init__.py
  ├── admin.py
  ├── apps.py
  ├── forms.py
  ├── models.py
  ├── recommend.py
  ├── serializers.py
  ├── tests.py
  ├── urls.py
  └── views.py

```

app

└ __pycache__

└ fixtures : json파일이 저장된 폴더 (연동된 DB에 json파일의 형태로 데이터 삽입을 하기 위한 폴더)

└ migrations : 연동된 DB의 수정된 작업들이 저장된 폴더 (테이블 생성, 수정, 삭제한 기록들이 저장된 폴더)

└ static : templates 폴더안의 html 파일들에 대한 css, js, img 파일들이 저장된 폴더, (static 폴더안에 파일을 저장해야 접근이 가능)

└ templates : html 파일들이 저장된 폴더

└ __init__.py

└ admin.py : Django Admin 사이트에서 사용할 모델 등록 파일

└ apps.py : 앱 설정 파일, 앱에 대한 설정을 수정할 때 사용한다

└ forms.py

└ models.py : 연동한 DB의 테이블이 저장된 파일

└ recommend.py : 장르 기반 추천 알고리즘이 저장된 파일

└ serializers.py

└ tests.py : 테스트 케이스를 작성하는 파일

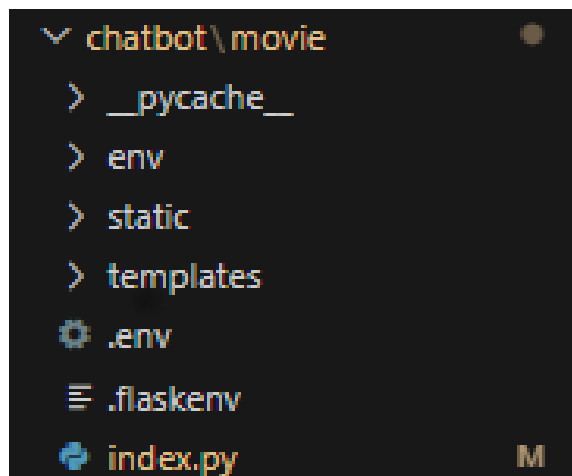
└ urls.py : 앱의 URL 패턴을 정의하는 파일

└ views.py : 앱의 뷰를 정의하는 파일 (urls.py파일에 작성하여 정의한 URL을 HTML파일과 이어주고, 사용할 데이터를 넘겨주는 파일)

프로젝트 수행내용

Flask

파일 내용 구성



Chatbot : flask 프로젝트의 루트 폴더

└ __pycache__

└ env : 가상환경에 대한 설정 정보가 담긴 폴더

└ static : HTML 파일 외에 img, css와 같은 파일을 담아두는 폴더

└ templates : HTML 파일을 담아두고 불러오는 폴더

└ .env : 챗봇 구현을 위해 TMDB, Dialogflow, Pusher를 사용하기 위한
TMDB_API_KEY, Dialogflow 프로젝트 ID, PUSHER의 정보가 저장된 파일

└ .flaskenv : 기본 설정 파일

└ index.py : templates 폴더의 HTML 파일을 불러오는 파이썬 파일, 이 파일에 작성된 함수, 지정한 라우팅을 이용하여 실행된다.

* 진행하는 프로젝트는 Django인데 챗봇은 Flask로 구현한 이유 ?

Flask는 Django와 같은 웹 프레임워크이다.

Django : Full-stack Framework / Flask : Micro Framework

Flask는 가벼운 프레임워크이기 때문에 원하는 기능을 편하게 확장할 수 있으며 보다 유연하다.

Django는 Flask보다 10배는 무겁고 자유도가 낮기 때문에 Chatbot은 Flask를 이용하여 구현하였다.

Django는 한 프로젝트 내에 다양한 앱이 존재할 수 있지만 Flask는 1개의 앱을 개발하도록 되어있다.

진행중인 장고 프로젝트와 별개로 챗봇 구현을 진행하고 유지보수 하였기 때문에 보다 자유도가 높은 Flask를 이용하여 챗봇 구현을 진행하였고 각각 서버를 따로 구동시켜 URL로 두 프로젝트를 연결하는 방식으로 진행하였다.

메뉴별 기능 [로그인 창]



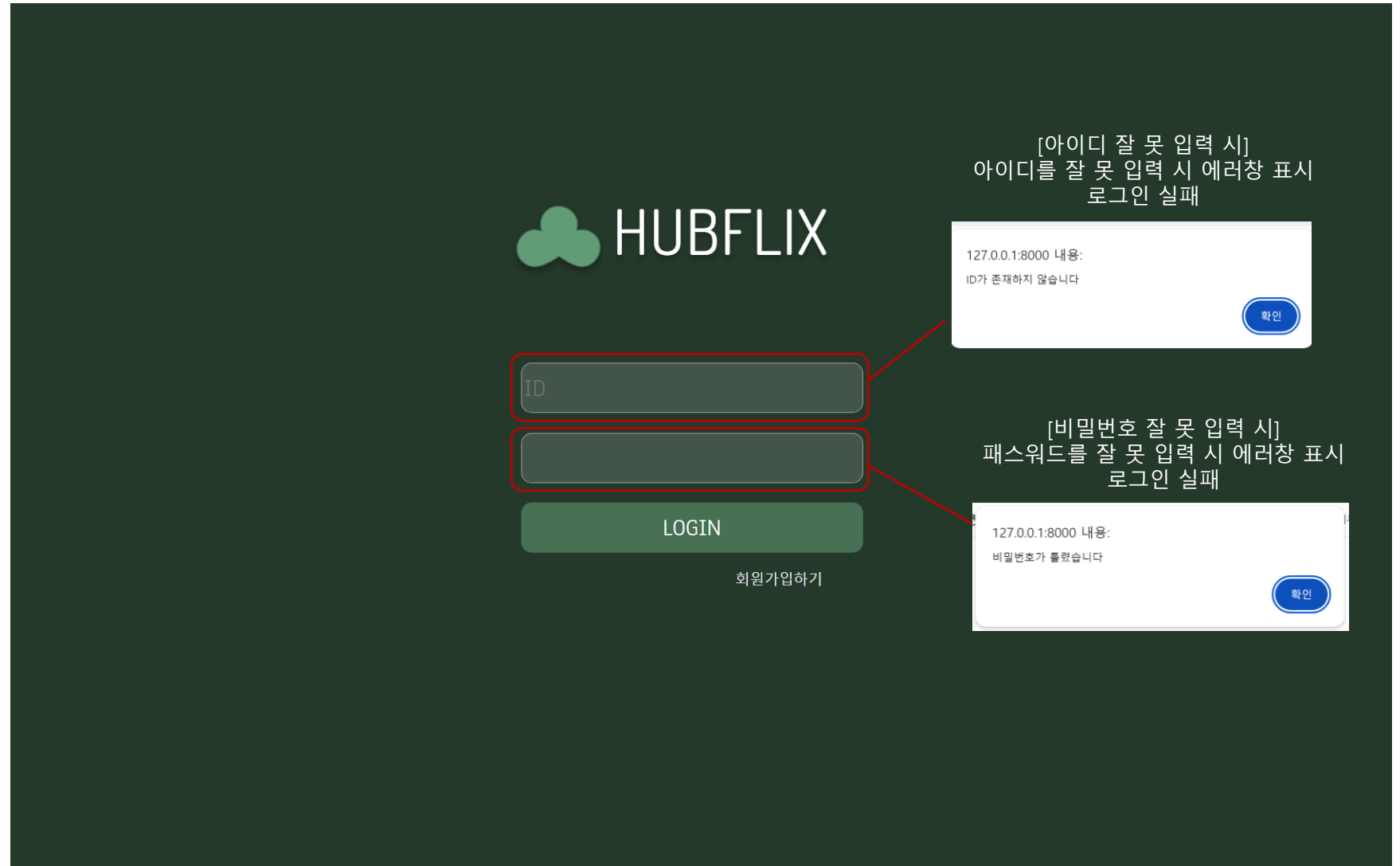
[아이디,패스워드 입력창]
등록한 아이디와 패스워드
를 입력하여 로그인

LOGIN

회원가입하기

[회원가입]
클릭시 회원가입 창으로 이동

메뉴별 기능 [로그인 창]



메뉴별 기능

[회원가입창]



회원가입

아이디

ex) abc123

이메일

ex) abc123@naver.com

비밀번호

영문, 숫자 조합 8~16자

비밀번호 확인

닉네임

ex) 홍길동

생년월일

2024-01-01



완료

프로젝트 수행내용

메뉴별 기능 [회원가입창]

아이디
jjaenni

이메일
hyoun3842@naver.com

비밀번호
.....

비밀번호 확인
.....

닉네임
채은

생년월일
2002-05-21

완료

[이미 존재하는 id일 경우]
이미 존재하는 id 일시 에러 창 표시
회원가입 실패

127.0.0.1:8000 내용:
ID가 이미 존재합니다

확인

[이메일 형식이 아닐 경우]
이메일 형식이 아닐 시 에러 창 표시
회원가입 실패

이메일

hyoun384

127.0.0.1:8000 내용:
이메일 형식이 틀렸습니다

확인

[비밀번호 불일치일 경우]
비밀번호가 불일치할 시 에러 창 표시
회원가입 실패

127.0.0.1:8000 내용:
비밀번호가 다릅니다

확인

메뉴별 기능 [회원가입창]

아이디
jjaenni

이메일
hyoun3842@naver.com

비밀번호
.....

비밀번호 확인
.....

닉네임
채은

생년월일
2002-05-21

완료

[이미 존재하는 닉네임일 경우]
이미 존재하는 닉네임일 시 에러 창 표시
회원가입 실패

127.0.0.1:8000 내용:
닉네임이 이미 존재합니다

확인

2002년 05월 ▾ ↑ ↓

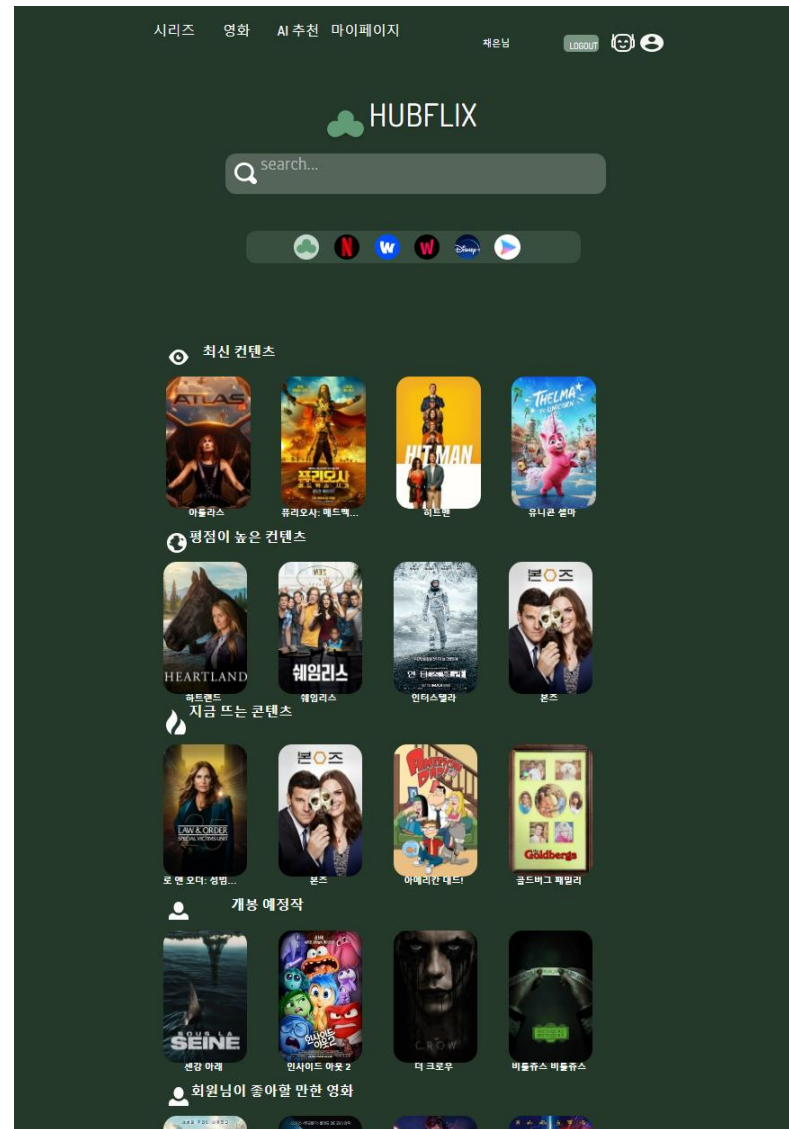
일	월	화	수	목	금	토
28	29	30	1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	1
2	3	4	5	6	7	8

삭제 오늘

2002-05-21

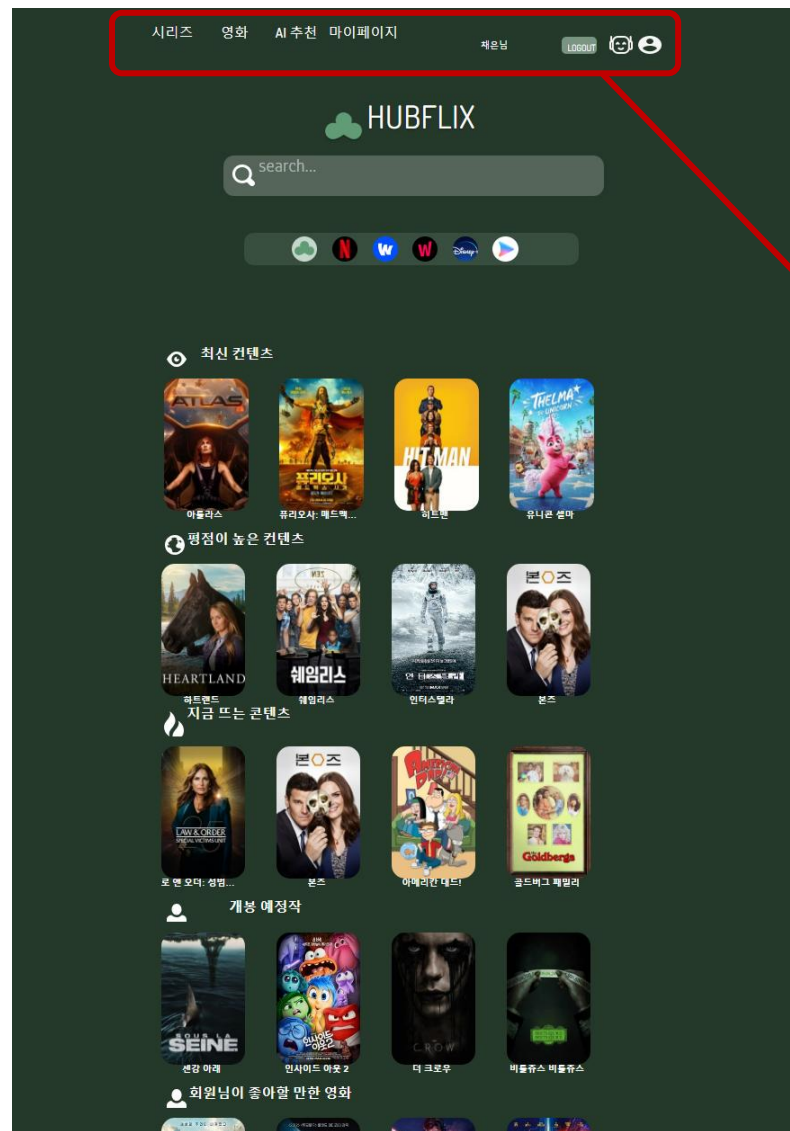
[생년월일]
클릭 시 캘린더를 통해 생년월일 선택 가능

메뉴별 기능 [메인창]



로그인 성공시
메인하면 창으로 이동

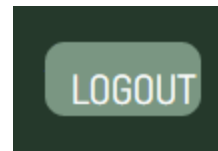
프로젝트 수행내용

메뉴별
기능
[메인창]

[카테고리]
메인창 상단에 카테고리를 두어
시리즈/영화별로 콘텐츠 표시

시리즈 영화 AI 추천 마이페이지

[logout icon]
Logout icon클릭시
계정 로그아웃



[AI챗봇 icon]
AI챗봇icon클릭시
AI챗봇 창으로 이동

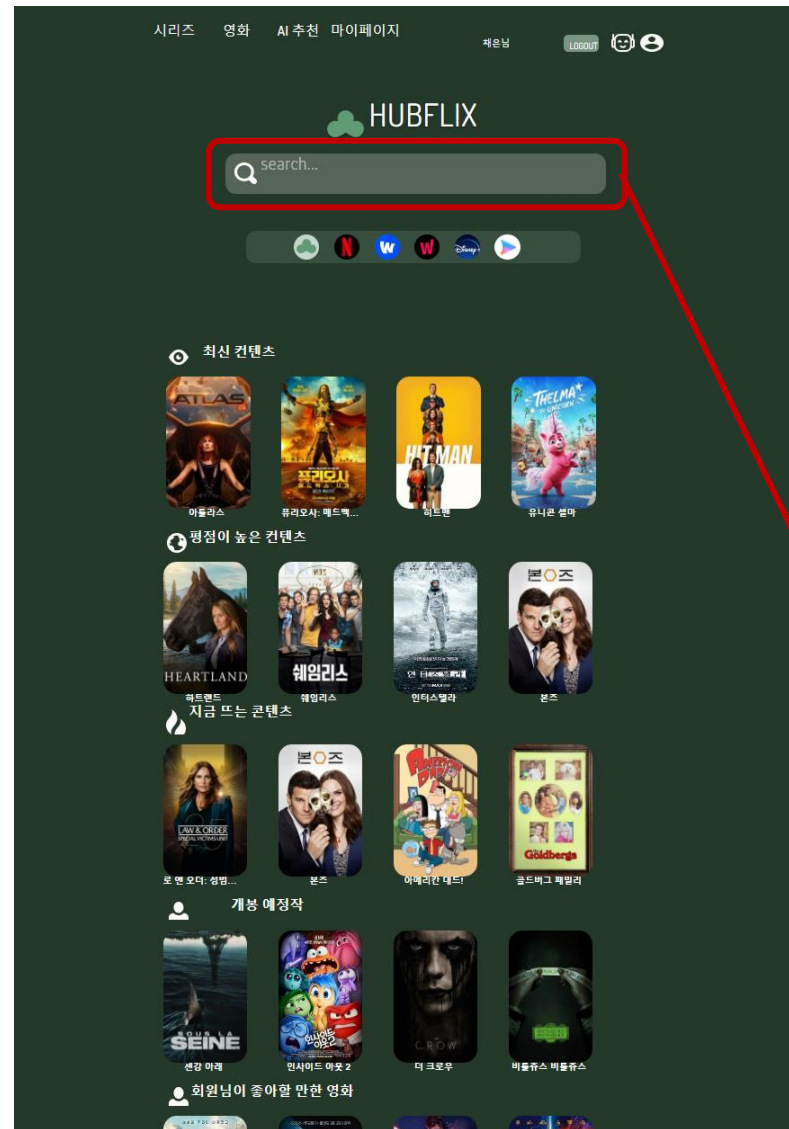


[마이페이지 icon]
마이페이지 icon클릭시
마이페이지 창으로 이동

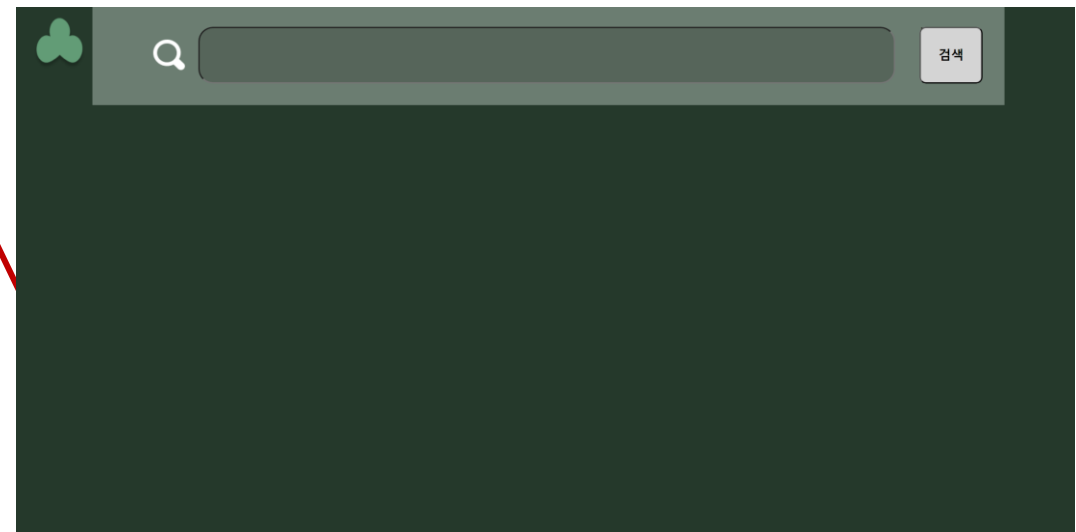


프로젝트 수행내용

메뉴별 기능 [메인창]



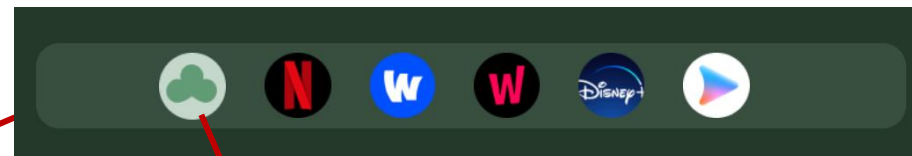
검색바 클릭 시 콘텐츠 검색창으로 이동



[콘텐츠 검색창]

프로젝트 수행내용

[OTT 선택 카테고리]



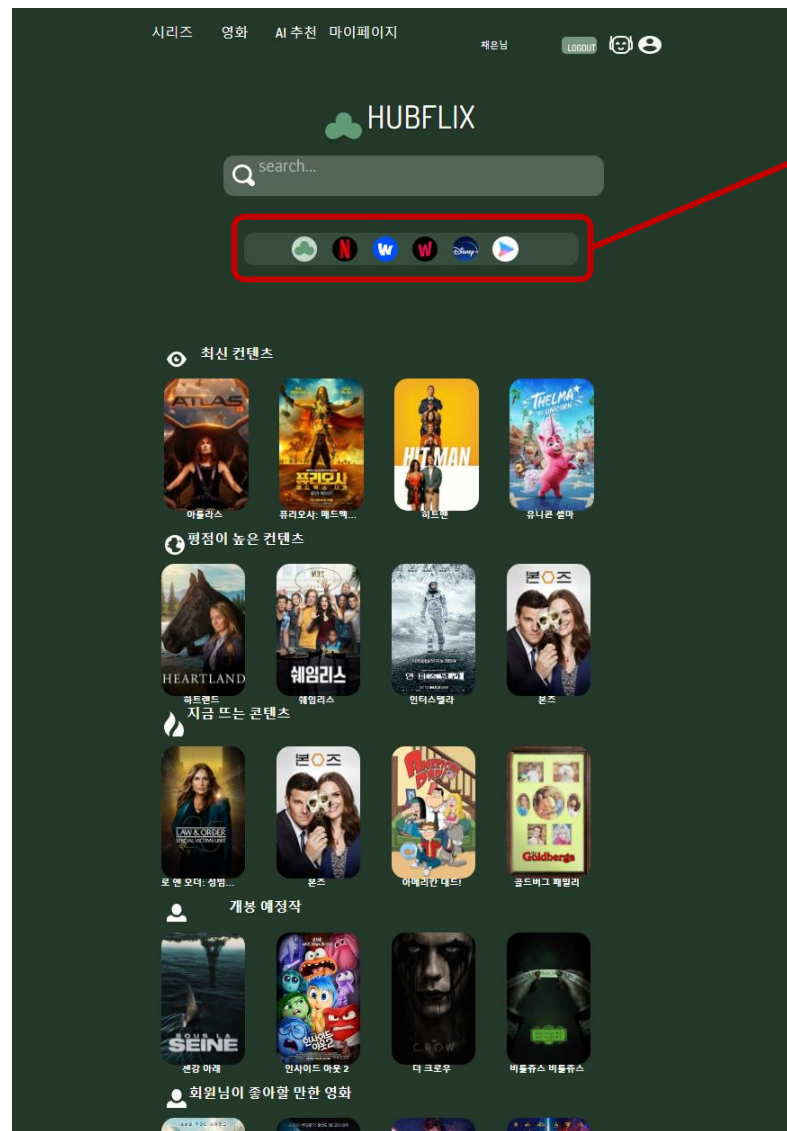
각각의 OTT 클릭 시
클릭한 OTT에 해당하는 콘텐츠만 메인화면에 표시

[OTT통합 icon]



해당 아이콘 클릭시
모든 OTT 서비스를 통합하여 콘텐츠
가 메인화면에 표시됨

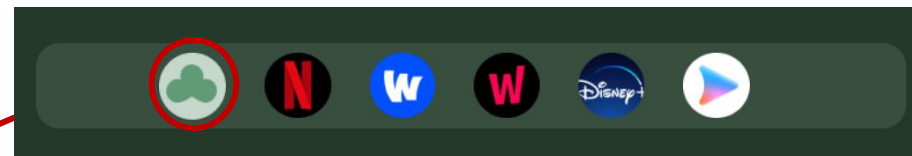
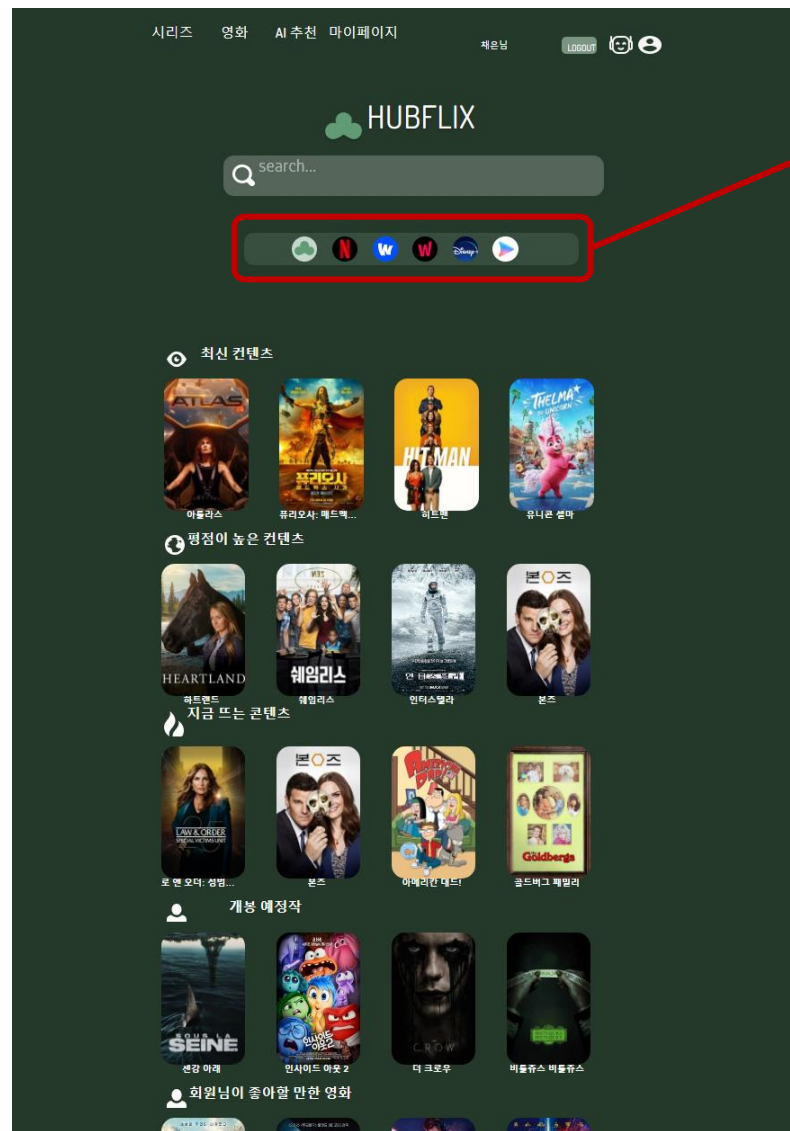
메뉴별 기능 [메인창]



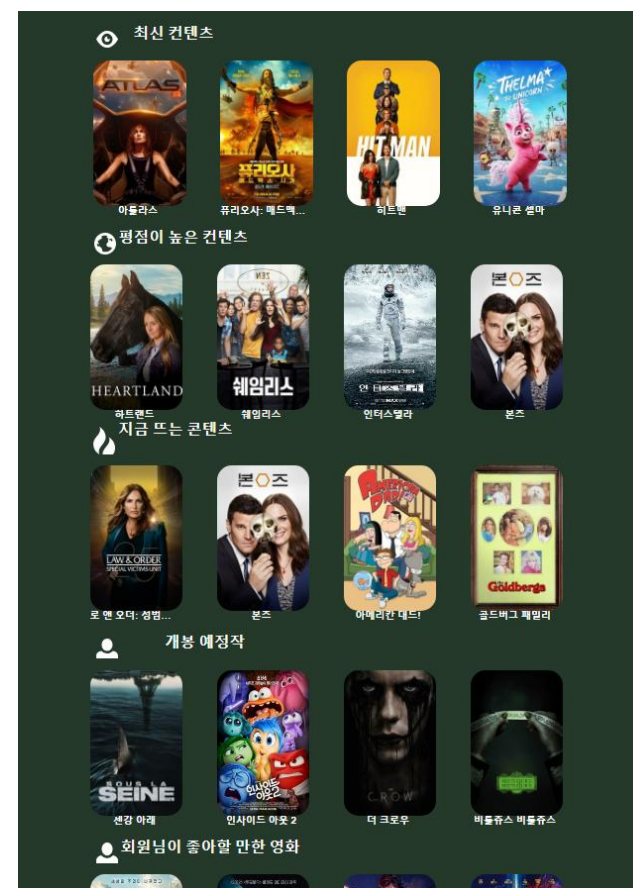
프로젝트 수행내용

[hubflix 아이콘 클릭 시]

메뉴별 기능 [메인창]



Hubflix 아이콘 클릭 시
Hubflix에 있는 모든 OTT서비스의 콘텐츠들로 정렬

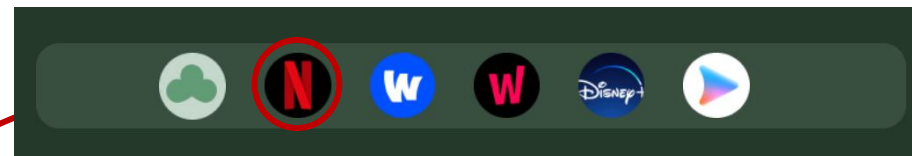
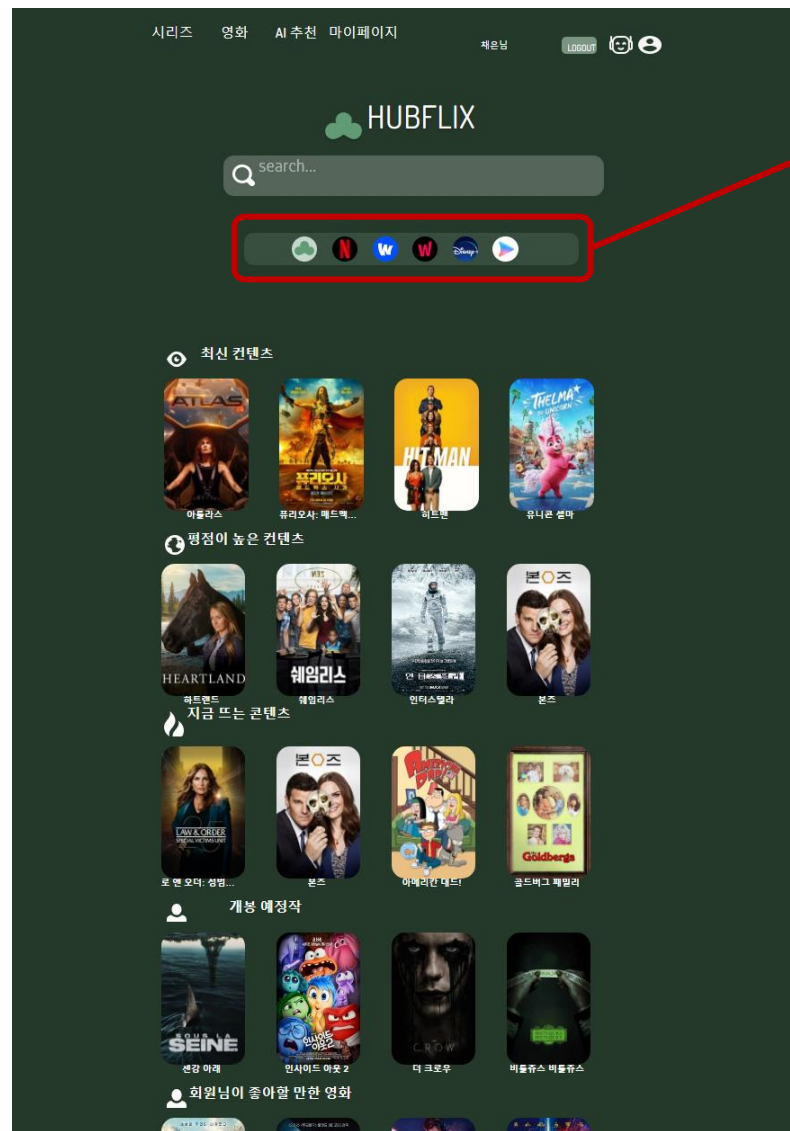


[모든 OTT서비스 통합 콘텐츠
들]

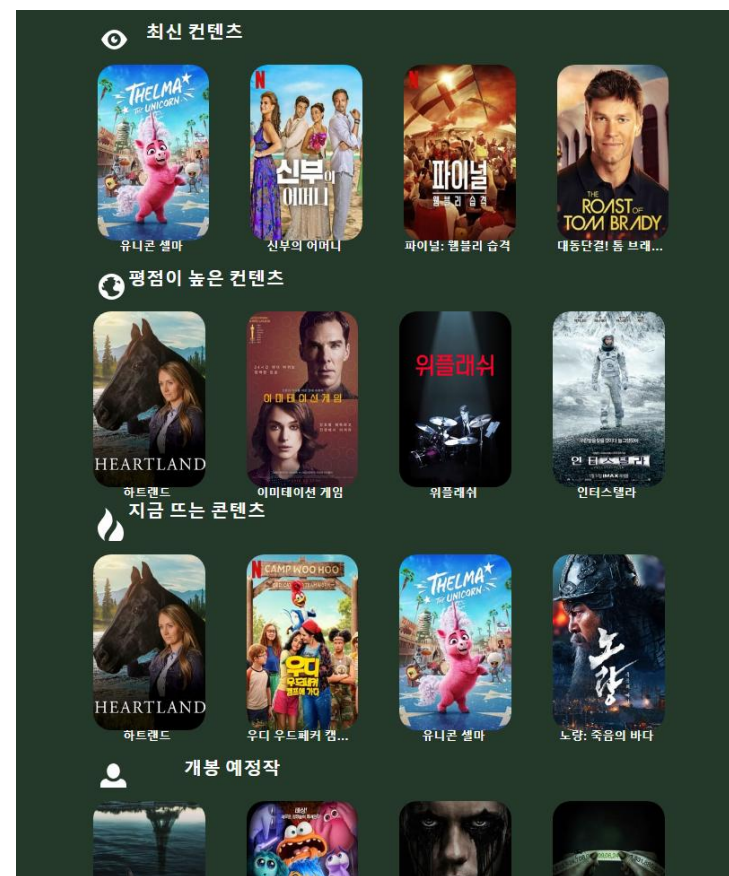
프로젝트 수행내용

[넷플릭스 아이콘 클릭 시]

메뉴별 기능 [메인창]



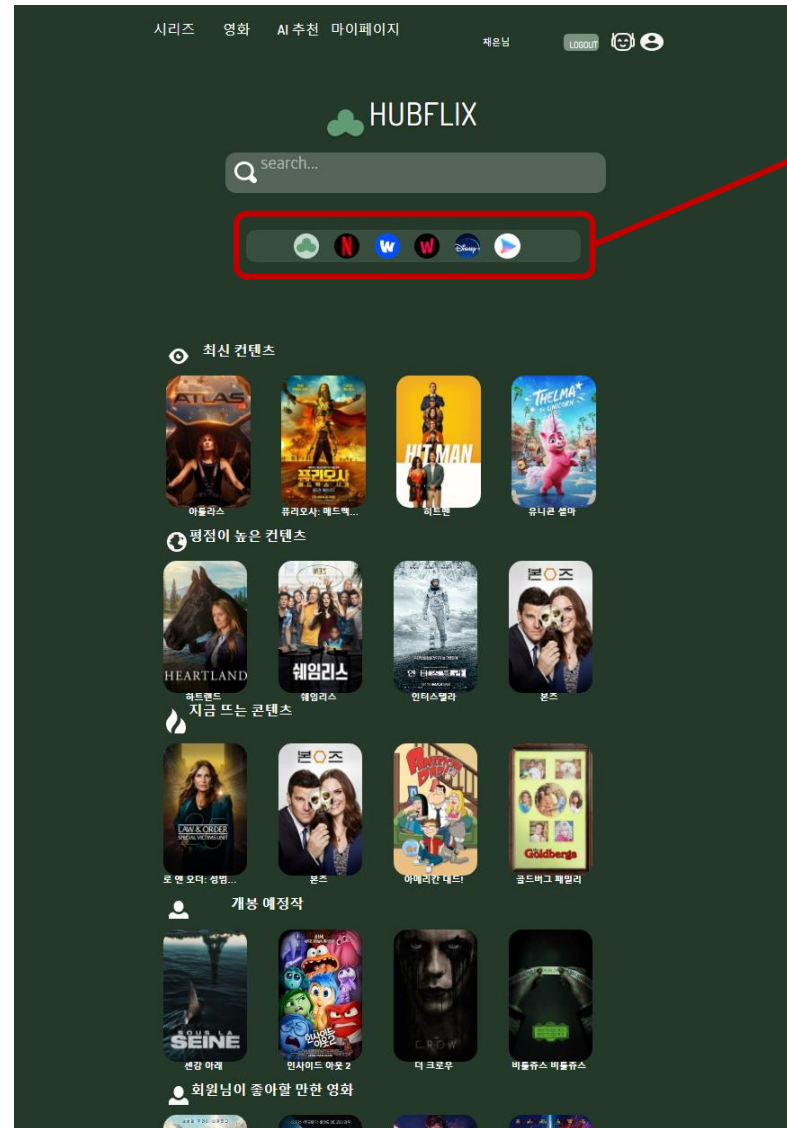
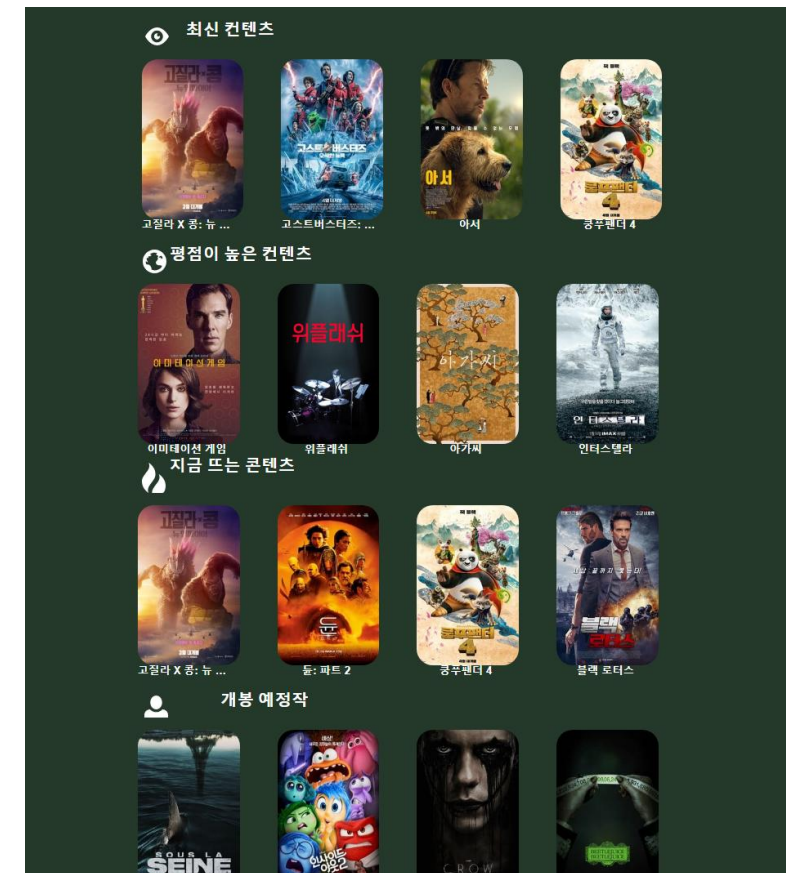
넷플릭스 아이콘 클릭 시
넷플릭스에 해당하는 OTT콘텐츠 들로만 정렬됨



[넷플릭스에 해당하는 콘텐츠
들]

프로젝트 수행내용

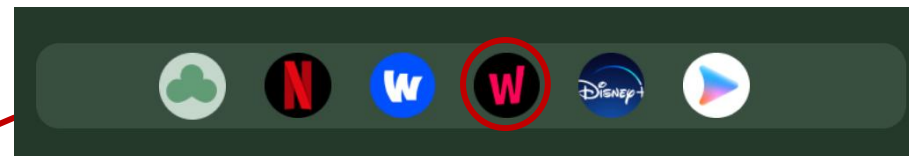
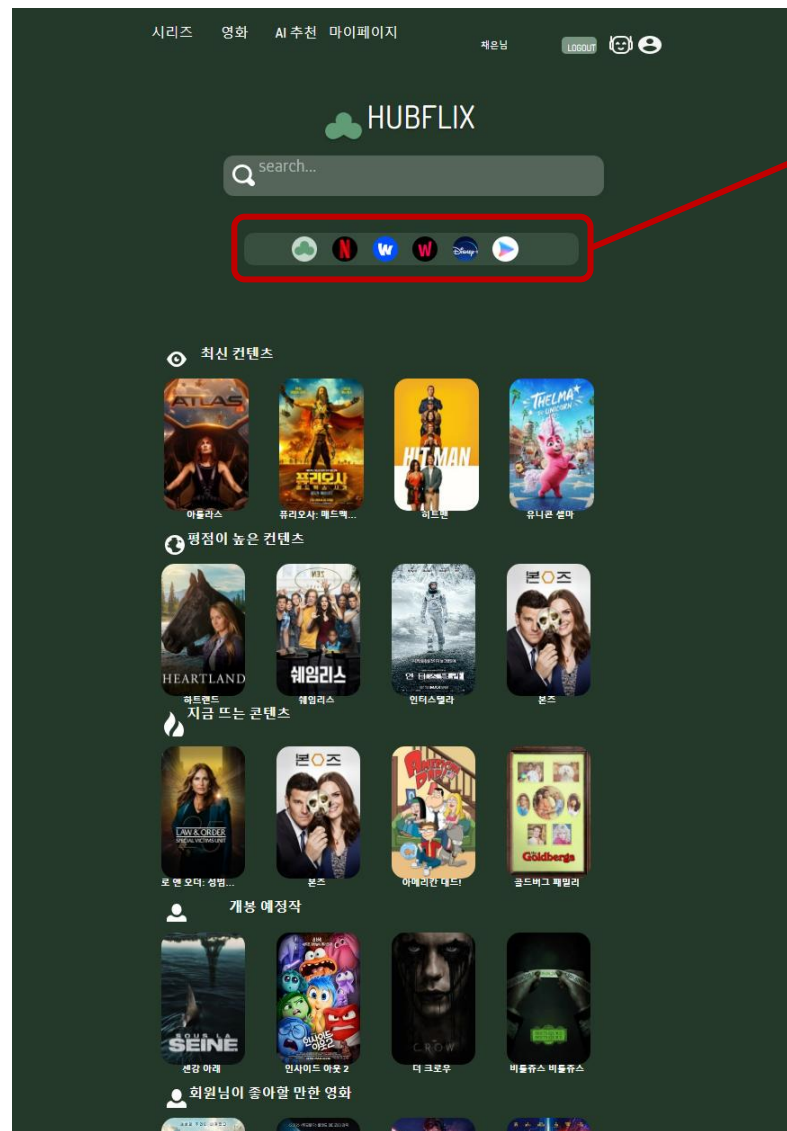
[웨이브 아이콘 클릭 시]

메뉴별
기능
[메인창]웨이브 아이콘 클릭 시
웨이브에 해당하는 OTT콘텐츠 들로만 정렬

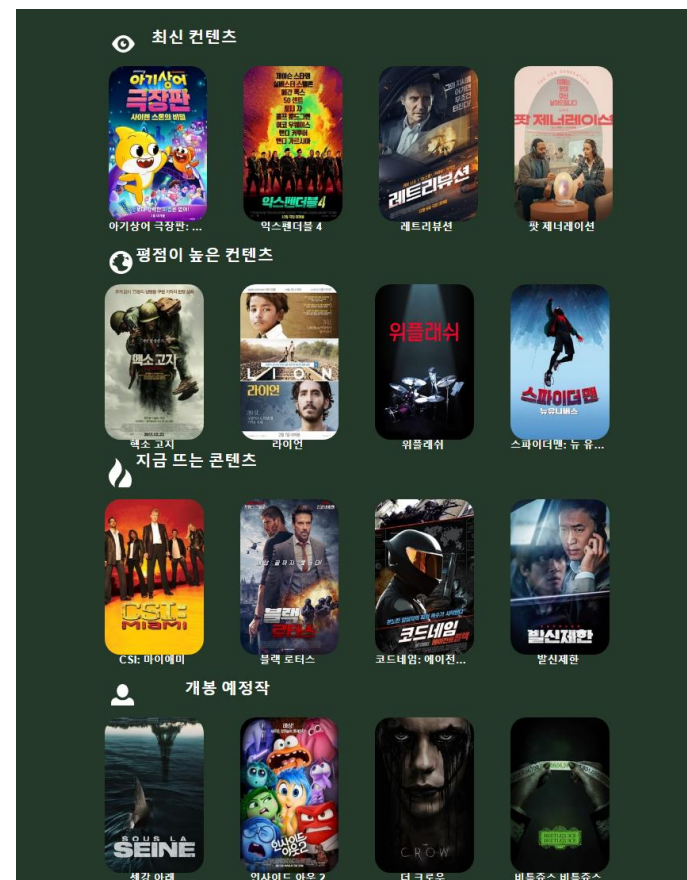
[웨이브에 해당하는 콘텐츠들]

프로젝트 수행내용

[왓차 아이콘 클릭 시]

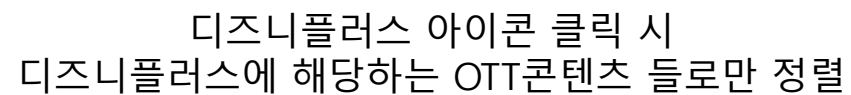
메뉴별
기능
[메인창]

왓차 아이콘 클릭 시
왓차에 해당하는 OTT콘텐츠 들로만 정렬됨

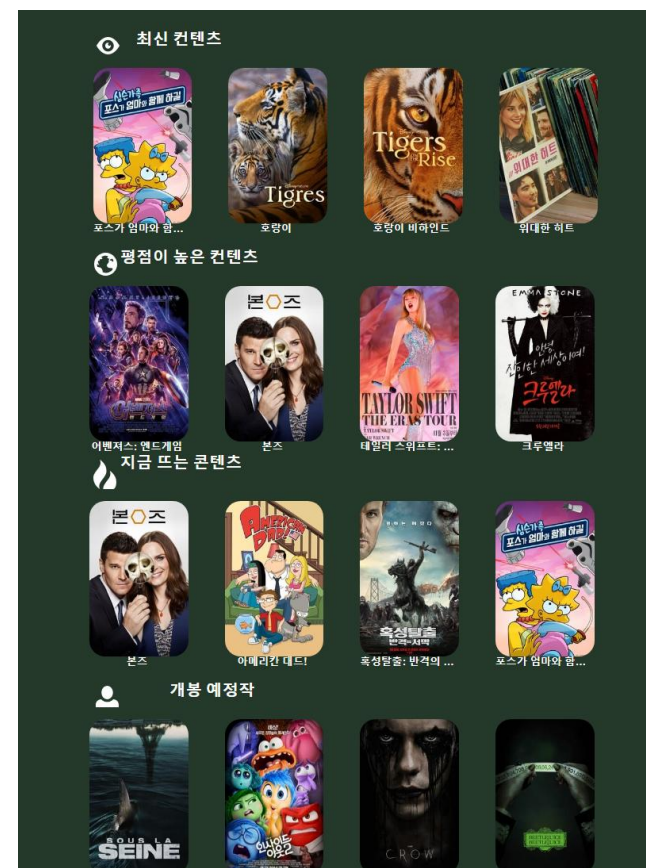


[왓차에 해당하는 콘텐츠들]

[디즈니플러스 아이콘 클릭 시]



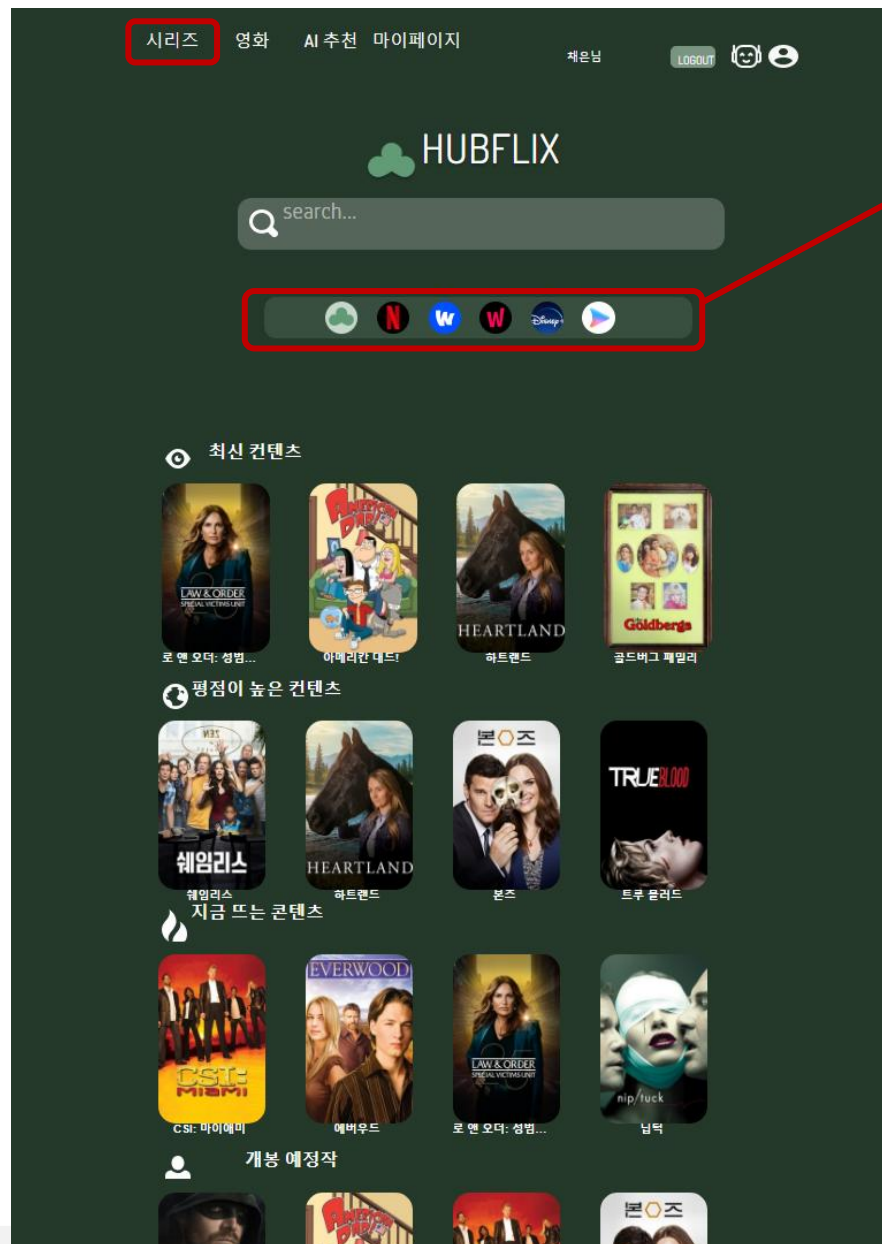
메뉴별 기능 [메인창]



[디즈니플러스에 해당하는 콘텐츠들]

프로젝트 수행내용

메뉴별 기능 [시리즈 창]

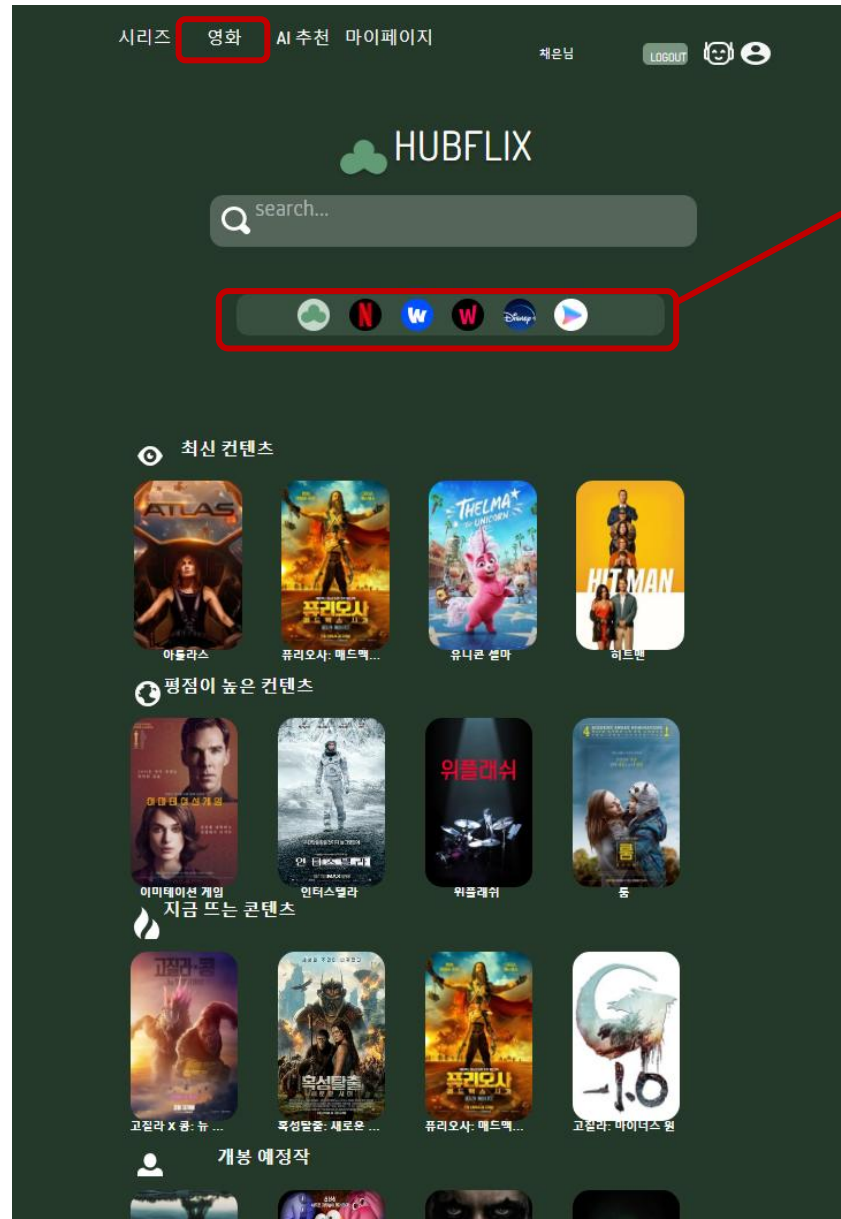


[OTT 선택 카테고리]

메인창과 마찬가지로 각각의 OTT 클릭 시
클릭한 OTT에 해당하는 콘텐츠만 메인화면에 표시
됨

프로젝트 수행내용

메뉴별 기능 [영화 창]



[OTT 선택 카테고리]



메인창과 마찬가지로 각각의 OTT 클릭 시
클릭한 OTT에 해당하는 콘텐츠만 메인화면에 표시
됨

프로젝트 수행내용

메뉴별 기능

[상세정보 창
-작 정보]

[보기 icon]



Icon 클릭 시
검색창으로 이동

[hubflix icon]



Icon 클릭 시
뒤로가기
전페이지로 이동

2023-12-18

tv

애니메이션,코미디

15세이상시청가

아메리칸 대드!

[작품정보 태그ver]
개봉일,장르,시리즈/영화,연
령대를 태그 형식으로 표시

랭글리 폴즈라는 도시에서 사는 CIA에
몰에 과대망상이 매우 심한 아버지(스탠 스미스)와 전
형적인 금발백치 어머니(프랜신 스미스), 반항아에 좌
빨인 장녀(헤일리 스미스)에 Nerd 아들(스티브 스미
스), 변장을 좋아하는 양성애자자웅동체[1] 외계인(로
저), 그리고 동독 스키점퍼의 기억을 가지고 있는 금붕
어(클라우스 하이슬러)로 이루어진 스미스 가족과 그
주변인물들을 다루고 있다

작품정보

보는 곳

장르	애니메이션,코미디
러닝타임	None분
개봉일	2023-12-18
평점	6

[작품정보]
작품 정보 클릭 시
콘텐츠의 장르,러닝타임,개봉일,평
점 등 콘텐츠 정보를 한눈에 수 있
음

프로젝트 수행내용

메뉴별 기능 [상세정보 창 -보는 곳]

HUBFLIX

2023-12-18 TV 애니메이션, 코미디 15세이상시청가

아메리칸 대드!

랭글리 폴즈라는 도시에서 사는 CIA에서 근무하는 수꼴에 과대망상이 매우 심한 아버지(스탠 스미스)와 전형적인 금발백치 어머니(프렌신 스미스), 반항아에 좌빨인 장녀(헤일리 스미스)에 Nerd 아들(스티브 스미스), 변장을 좋아하는 양성애자자웅동체[1] 외계인(로저), 그리고 동독 스키점퍼의 기억을 가지고 있는 금붕어(클라우스 하이슬러)로 이루어진 스미스 가족과 그 주변인물들을 다루고 있다

작품정보

보는 곳

보러가기

Classics	한국에서 지원하는 OTT서비스 없음	대여
Classics	한국에서 지원하는 OTT서비스 없음	구매
Disney	Disney Plus	구독

[보는 곳]
보는 곳 클릭 시
해당 콘텐츠가 해당하는 OTT를 표시

Disney+

이 모든 이야기가 여기에 지금 스트리밍 중

디즈니+ 스탠다드	디즈니+ 프리미엄
최대 1080p Full HD 비디오 최대 5:1 오디오 최대 2대 기기 동시 스트리밍	최대 4K UHD & HDR 비디오 최대 Dolby Atmos 오디오 최대 4대 기기 동시 스트리밍
월 ₩9,900	월 ₩13,900

(무가세 포함) (무가세 포함)

디즈니+ 프리미엄 연간 멤버십을 구독하고 최대 16% 할인*을 받으세요.
연간 멤버십을 포함한 멤버십 유형별 세부 정보를 확인해 보세요.

*월간 멤버십 12개월 구독료 대비 할인된 가격입니다. 추가 약관 적용.

[구독]
구독 클릭 시 해당 콘텐츠가 해당
하는 OTT사이트로 이동하여 바로
구독할 수 있도록함

프로젝트 수행내용

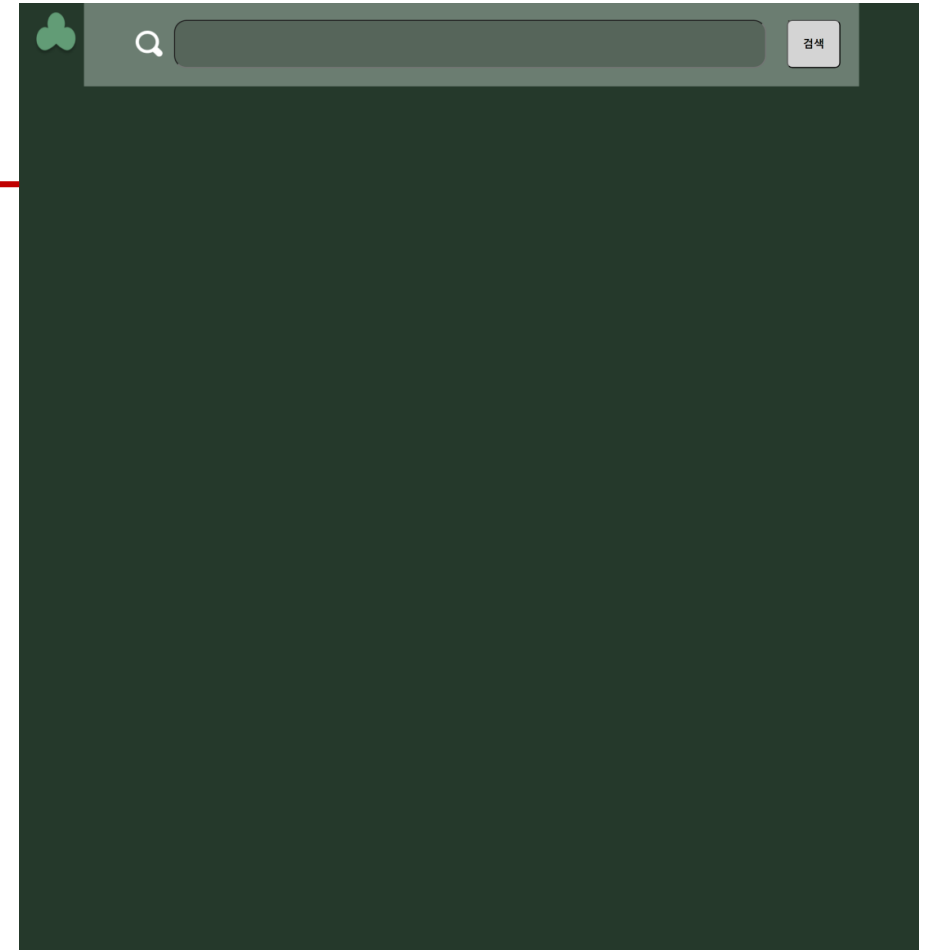
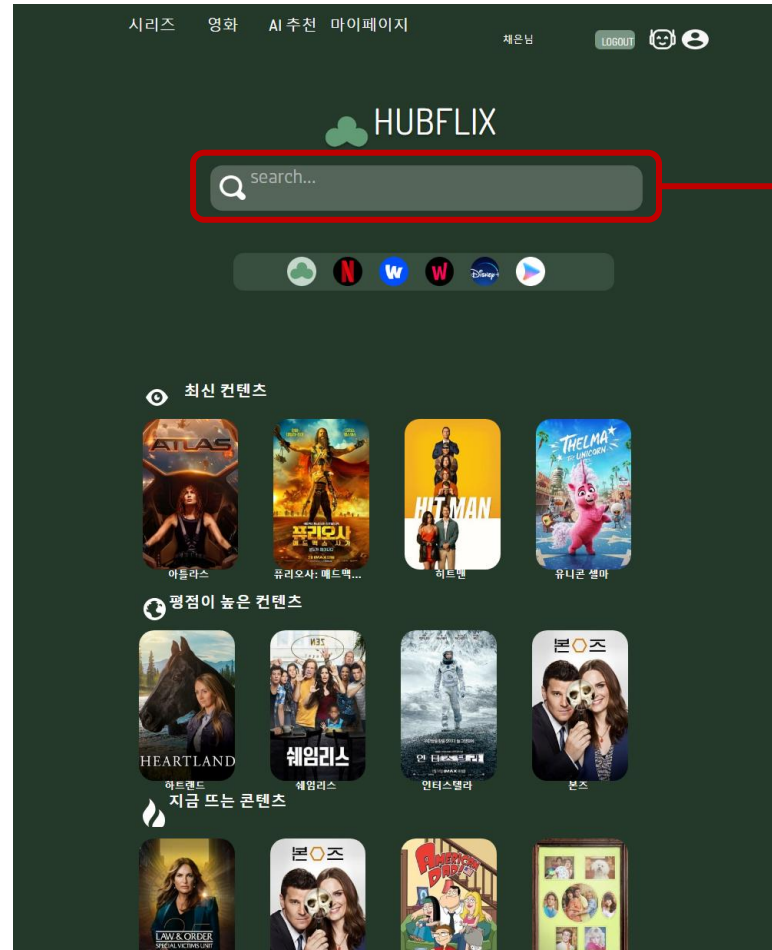
메뉴별 기능 [알고리즘]



원하는 콘텐츠를 클릭 시
해당 콘텐츠의 제목을 이용하여
장르를 기준으로 알고리즘이
적용되고 추천하는 콘텐츠가 바뀐다

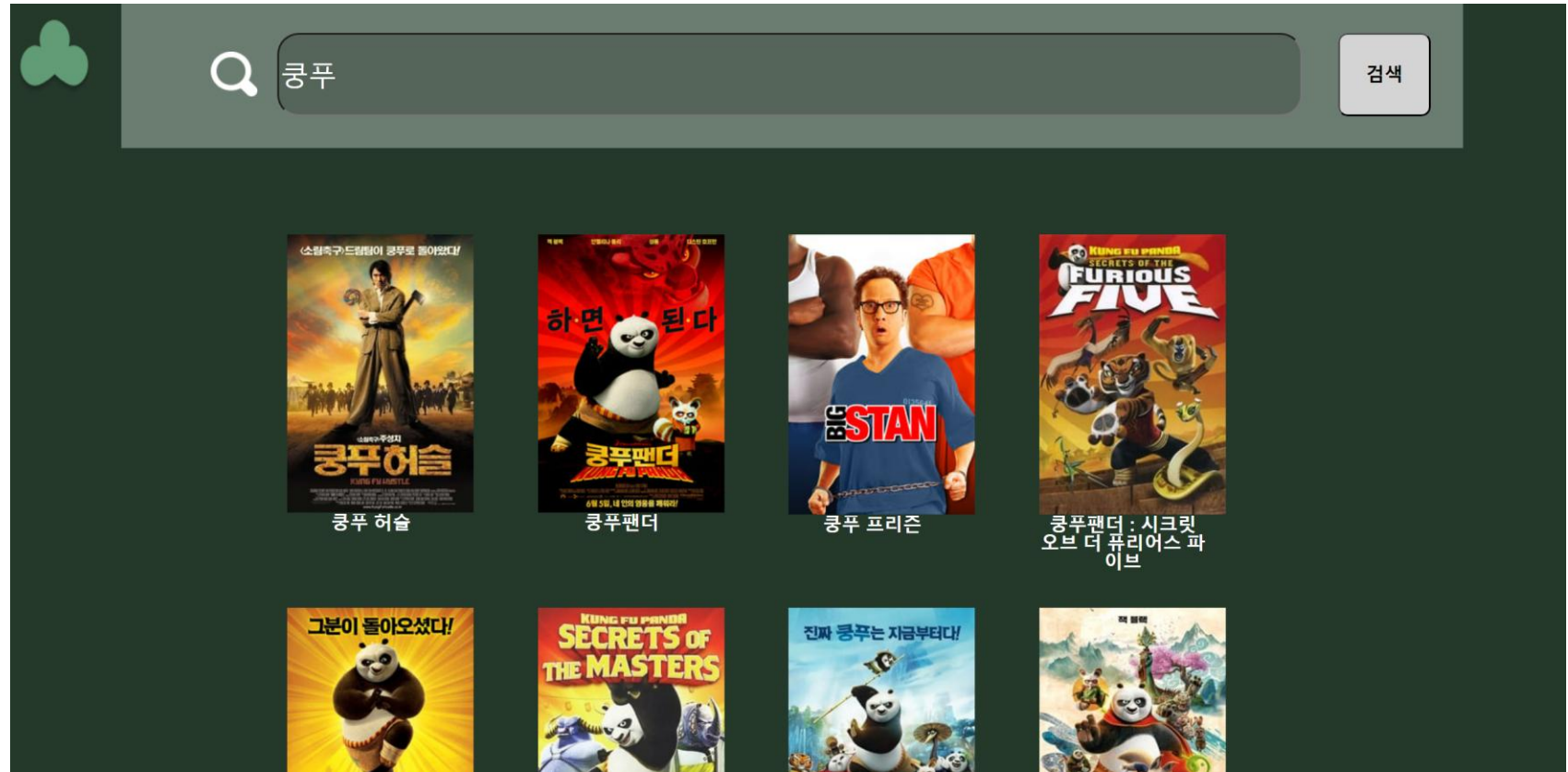


메뉴별 기능 [검색창]



메인창에서 검색바 클릭시 검색창 페이지로 이동

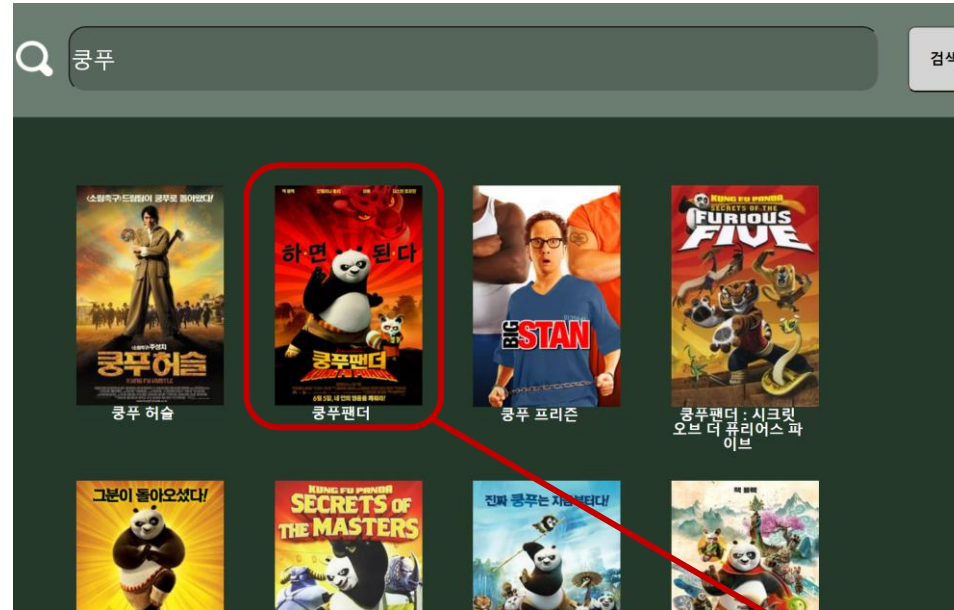
메뉴별 기능 [검색창]



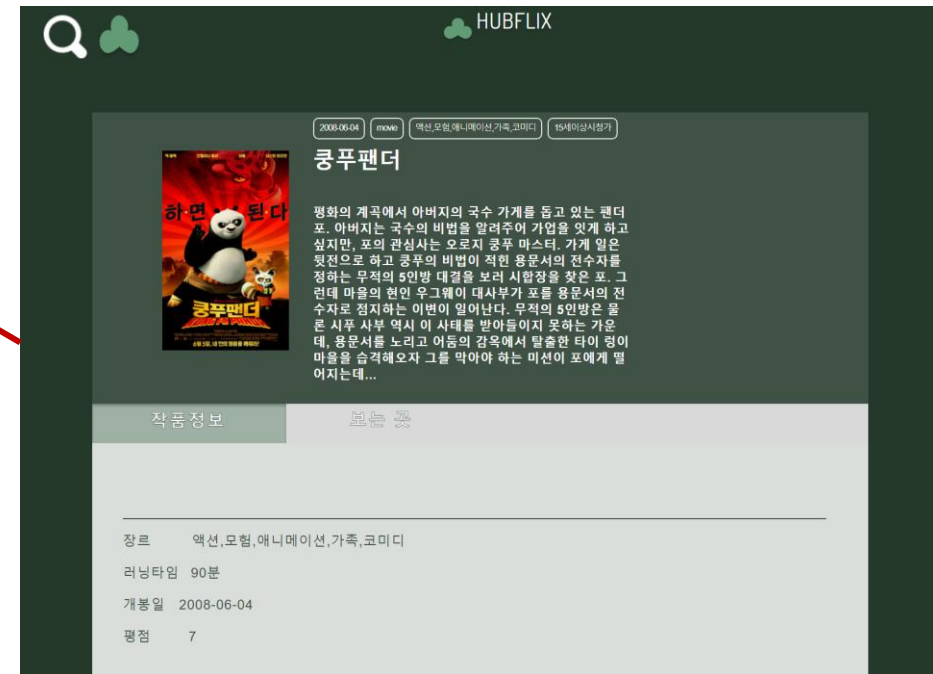
원하는 콘텐츠를 찾기위한 키워드를 검색하면
키워드가 포함된 콘텐츠들을 (모든 OTT 콘텐츠를 통합하여) 한눈에 볼 수
있음

프로젝트 수행내용

메뉴별 기능 [검색창]



원하는 콘텐츠를 클릭 시
해당 콘텐츠의 상세페이지로 이동



메뉴별 기능

[마이페이지
-나의 정보]



메뉴별 기능

[회원정보수정창]

HUBFLIX

정보 수정

아이디
jjaenni

이메일
hyoun3842@naver.com

닉네임
채은

생년월일
2024-01-01

완료

채은

조채

정보수정창에서 닉네임을 변경하면
마이페이지에서 변경된 닉네임을 확인할 수 있

HUBFLIX



조채

MY OTT

나의 정보

나의 정보

아이디 : jjaenni

닉네임 : 조채

이메일 : hyoun3842@naver.com

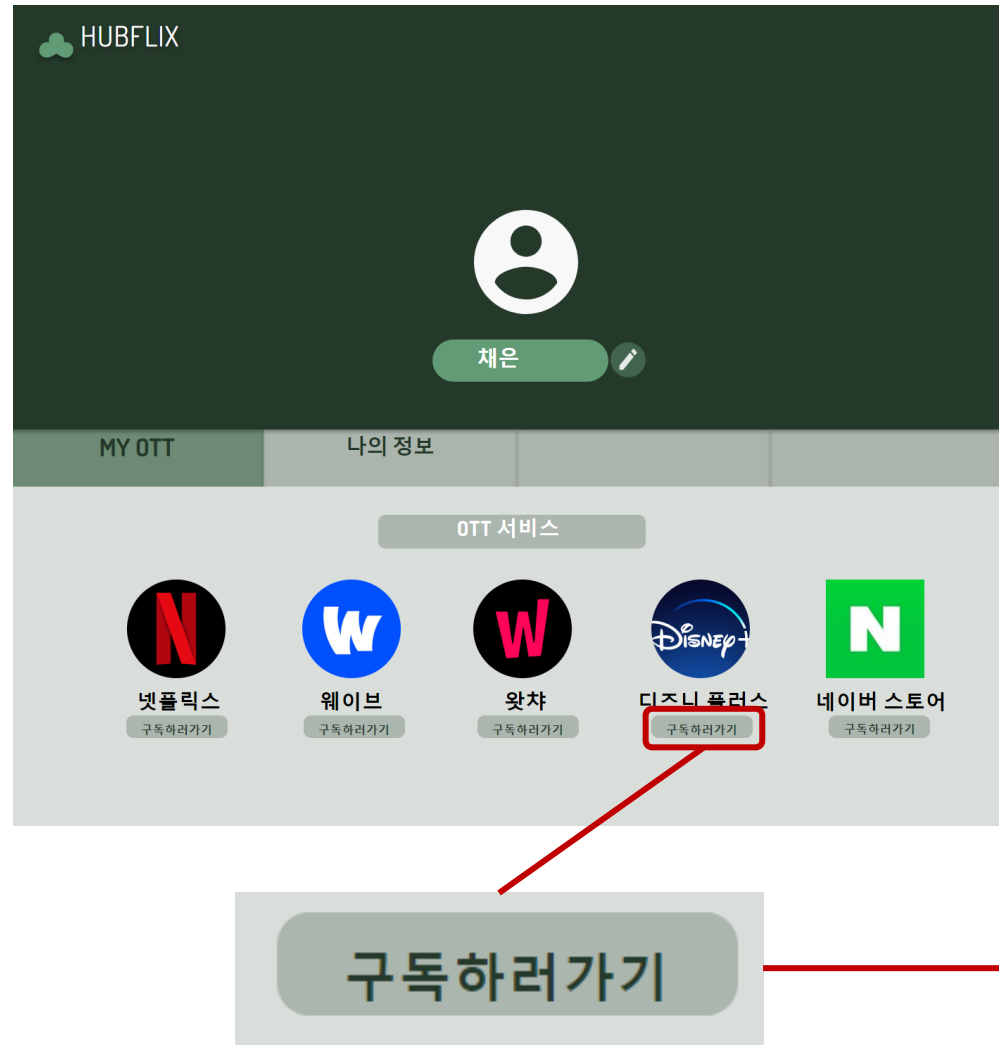
생일 : 2024-01-01

[회원정보수정]
기존의 아이디, 이메일, 닉네임, 생년월일을
수정할 수 있으며 완료버튼을 누르면 변경
됨

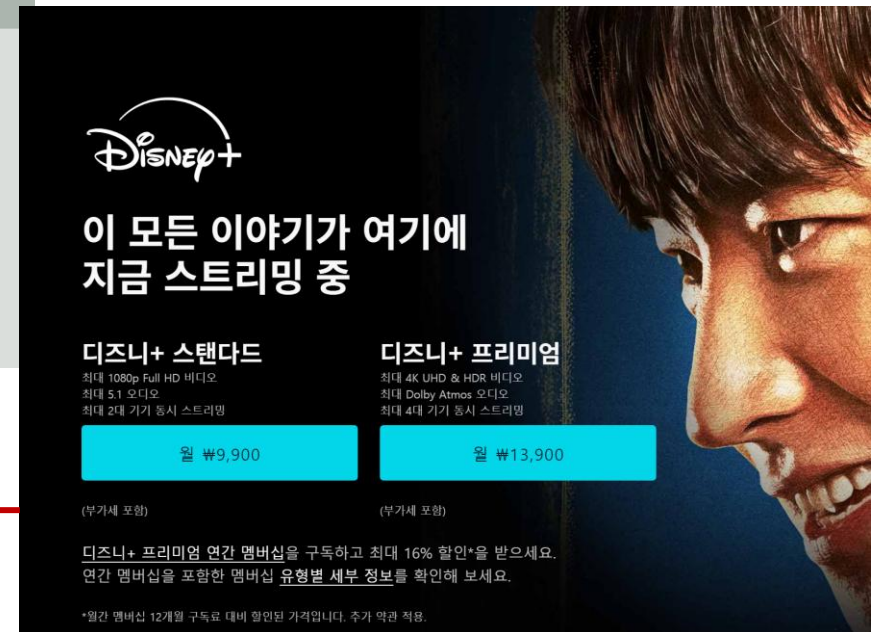
프로젝트 수행내용

메뉴별 기능

[마이페이지
-MY OTT]



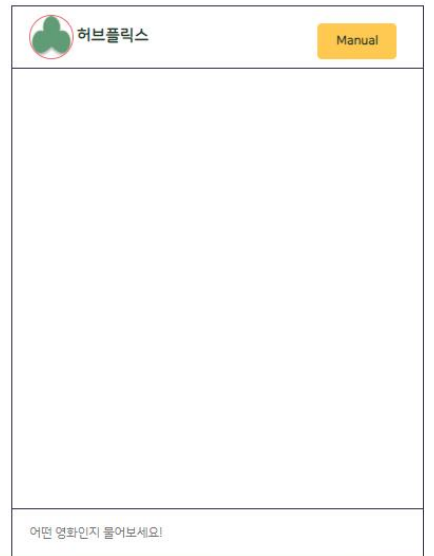
[MY OTT]
Hubflix에서 지원하는 모든OTT서비스를 한
눈에 볼 수 있으며 구독하러가기 버튼을 통
해
해당 OTT서비스의 플랫폼으로 이동하여
바로 구독할 수 있도록 함



프로젝트 수행내용

AI 챗봇 또는 챗봇 아이콘 클릭 시 챗봇 창으로 이동

메뉴별 기능 [챗봇]



프로젝트 수행내용

메뉴별
기능
[챗봇]

Manual 버튼 클릭 시



허브플릭스

Manual

다음은 검색 매뉴얼입니다.

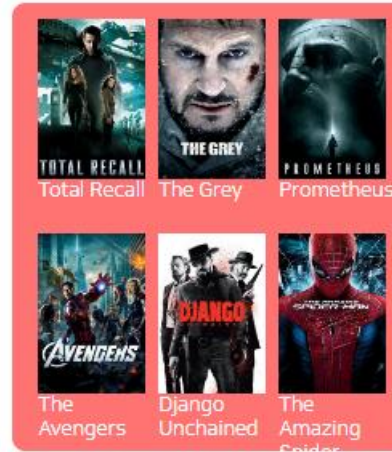
연도별 검색 : 2017년 영화 추천
 장르별 검색 : 액션 영화 추천
 영화 제목 검색 : love 검색
 인기 영화 검색 : 인기 영화 추천
 최근 영화 검색 : 최근 영화 추천

어떤 영화인지 물어보세요!

연도별 검색 : ex) 2012년 영화 추천 입력 시

2012년 영화 추천

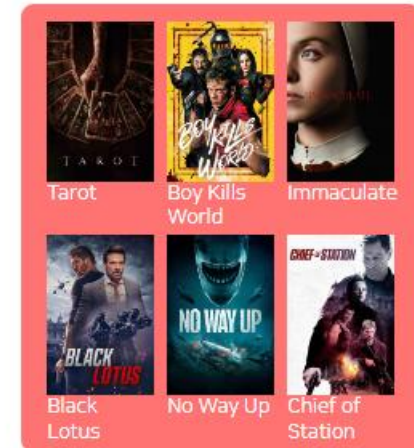
2012 에 개봉한 영화들 목록이에요



장르별 검색 : ex) 스릴러 영화 추천 입력 시

스릴러 영화 추천

스릴러 장르 영화를 추천해드릴게요



프로젝트 수행내용

메뉴별
기능
[챗봇]

영화 제목 검색 : ex) 쿵푸 검색 입력 시

쿵푸 검색

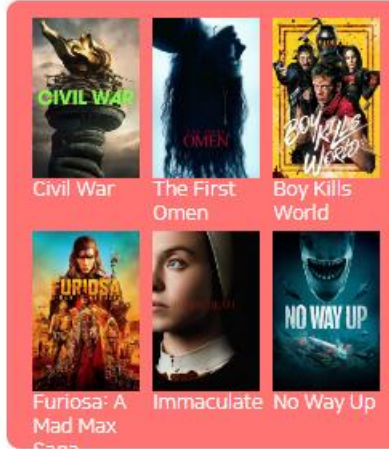
쿵푸 검색 결과는 다음과 같습니다.



인기 영화 검색 : ex) 인기 영화 추천 입력 시

인기 영화 추천

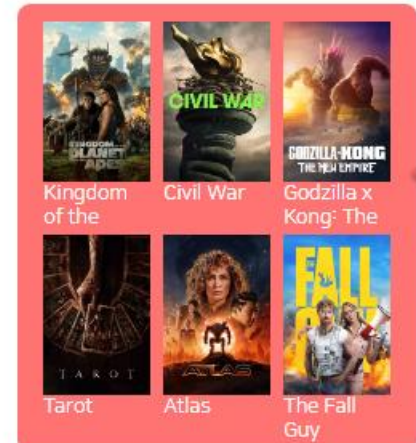
인기있는 영화를 추천해드릴게요



최근 영화 검색 : ex) 최근 영화 추천 입력 시

최근 영화 추천

지금 상영중인 영화를 추천해드릴게요



프로젝트 수행내용

메뉴별
기능
[챗봇]

검색 결과에서 포스터 이미지 클릭 시



Kung Fu Panda

Original-title Kung Fu Panda
 Original-nation United States of America
 Genre Action, Adventure, Animation, Family
 Release-date 2008-06-04
 Star-rating ★★★★★

martial arts # kung fu # strong woman # china # tiger

Crews



Mark Osborne



John Stevenson



Glenn Berger

Related Film



Derailed



Fist of Legend



Lone Wolf and Cub: Baby Cart at the River Styx

Cast Actor



Jack Black



Angelina Jolie



Dustin Hoffman



Kull the Conqueror



Slave Girls of Sheba



The Bloodletting

Plot

When the Valley of Peace is threatened, lazy Po the panda discovers his destiny as the "chosen one" and trains to become a kung fu hero, but transforming the unsleek slacker into a brave warrior won't be easy. It's

Related YouTube



AI챗봇기능

AI챗봇을 이용하여 사용자의 선호도와 행동을 분석하여 맞춤형 추천 알고리즘을 제공



기존의 OTT 통합 플랫폼과는 달리 AI 챗봇 기능을 콘텐츠를 추천 이를 통해 사용자들은 보다 편리하게 원하는 콘텐츠를 찾을 수 있도록 함

역할분담 및 수행내용



가나다(조장)

서버 구축, DB 설계, 구축,
AWS EC2-Gunicorn-Nginx 연동,
데이터 수집, 로그인, 회원가입, 회원수
정, 검색기능 구현, 추천 알고리즘 구현,
챗봇 구현, DB - 기능 - UI 연동

회의록,PPT제작,보고서작성, 발표



가나2(조원)

UI/UX설계 및 디자인,
기능 - UI 연결
PPT제작,보고서작성



가나3(조원)

서버 구축 AI 챗봇 구현, API 통합
보고서작성

느낀점

이름	내용
가나1	프로젝트 주제 선정 후 평소 접하지 않았던 웹과 관련된 주제를 선정하게 되어 웹에 대한 공부, 웹을 개발하기 위한 프레임워크에 대한 공부와 병행하면서 진행하여 어려움이 있었다. 원활한 프로젝트 진행을 위해 확실한 역할분담을 했어야했는데 조장으로써 역할분담을 확실히 하지 못한것 같아 아쉬움이 남는다. 프로젝트 마지막 단계에서 서버를 이용하여 장고프로젝트를 배포하는데에 오류가 나서 결국 배포를 하지 못하였는데 이 부분은 향후 보완해야할 부분이라고 생각한다. 프로젝트를 진행하면서 서버 구축부터 AWS EC2-Gunicorn-Nginx 연동, DB 설계, 구축, 가상환경 세팅, Django 프레임워크를 이용한 웹 개발, HTML, CSS, API를 이용한 데이터 수집, 웹페이지에서의 기능 구현, 추천 알고리즘 구현, 챗봇 구현까지 모두 접해보며 다양한 언어, 기술, 도구들을 사용할 수 있는 경험이 된 것 같다.
가나2	OTT 통합 플랫폼 웹사이트 구현 프로젝트에 참여하면서 UI/UX 설계 및 디자인, OTT 플랫폼 연동, 개인화 추천 알고리즘, TMDB를 통한 Open API 데이터 수집 등을 접해보고 사용자 중심의 직관적이고 깔끔한 인터페이스를 설계하고 다양한 기기에서 일관된 경험을 제공하는 반응형 디자인을 적용했으며, 여러 플랫폼의 API를 연동하고 데이터 일관성을 유지하는 방법을 배웠고, 사용자 데이터를 분석하여 적용한 추천 알고리즘을 접해보며, TMDB Open API를 활용해 데이터를 수집하고 관리하는 과정에서 실무적 기술을 익히는 등 기술적 역량과 문제 해결 능력을 크게 향상시킬 수 있었다.
가나3	프로젝트에 참여하면서 실제로 동작하는 웹에 대한 이해와 서버 작동 등에 대해 연구 및 공부할 수 있는 기회가 되었다. 프론트에서 하는 일, 기능 및 서버에서 하는 일에 대해 구분할 수 있었고, 그 외에 기능을 추가하기 위해서는 어떤 과정을 거쳐야 하는지 알 수 있게 되었다. 직관적인 UI/UX에 뒷받침해서 기능들을 구현하는 점이 쉬운 단계는 아니었으나, 공부하며 진행할 수 있는 기회가 되었다. 데이터 수집부터 기능, 동작까지 한 번에 서비스가 구현되는 것을 경험해볼 수 있었고, Django라는 새로운 프레임워크를 배울 수 있는 기회가 되었다.



개인화 추천알고리즘 구현

맞춤형 추천 알고리즘을 통해 사용자에게 개인화된 서비스를 제공함으로써 사용자 경험을 크게 향상시키고 사용자들은 더욱 효율적으로 콘텐츠를 찾고 시청할 수 있음



대화형 AI 챗봇 구현

AI챗봇을 이용하여 맞춤형 추천 콘텐츠, 검색 결과 정보 제공

기본 기능 구현

기본적인 로그인, 회원가입, 회원수정, 검색 기능 구현

OTT 정보 제공

선택한 콘텐츠의 제공중인 OTT 서비스 정보 제공

AI 챗봇 추가 학습을 통한 기능 향상

사용자 피드백 수집 및 개선

커뮤니티 기능 활성화

데이터의 범위 증가

서버를 이용한 배포

추천 알고리즘 강화

QnA



감사합니다